950 Q&A Series • Exactly 40/80 Questions

Next.Js

40/80 Interview Questions & Answers

Next.js vs React | Pages Router vs App Router | File-based Routing | Dynamic Routes | Catch-all Routes | Route Groups | Parallel Routes | Intercepting Routes | Layouts vs Templates | Nested Layouts | loading.tsx | error.tsx | not-found.tsx | Link Component | useRouter / usePathname / useSearchParams | next/image Optimization | next/font Optimization | public Folder Usage | next.config.js | Environment Variables | Middleware | React Server Components (RSC) | Client vs Server Components | Streaming & Suspense | Hydration & Hydration Mismatch | getStaticProps | getServerSideProps | getStaticPaths | generateStaticParams | generateMetadata | Data Fetching in Server Components | Client-side Fetching | Parallel Fetching | Request Deduplication | Cache & Revalidation | Route Handlers | API Routes (App Router) | CORS Handling | Code Splitting | Dynamic Imports (next/dynamic) | Prefetching vs Preloading | Core Web Vitals | SEO & Metadata | Open Graph | Image & Font Optimization | Deployment (Vercel vs Docker) | Internationalization (i18n) | Testing Next.js Apps

→ All 40/80 questions answered with production code ←

→ Follow for Next: → Complete DSA Sheet to Crack Coding Interviews ←

■ Save • ■ Comment 'JS950' if you want the JS950 Interview Q&A • ■■ Repost

Kaushal Singh

Full-Stack Dev | MERN + React Native | Next.Js

1M+ Impressions | Follow for MERN · Next.Js · React Native · Interview Prep

Next: Complete DSA Sheet to Crack Coding Interviews | Comment 'JS950'

Q1 What is Next.js? Why use it over plain React?

■ Answer

Next.js is a React framework by Vercel that adds server-side rendering, routing, and full-stack capabilities. React alone is a UI library — Next.js is the complete production framework on top of it.

What Next.js adds to React:

- File-based routing — no react-router-dom needed
- Server-Side Rendering (SSR) — HTML on first load
- Static Site Generation (SSG) — pre-built HTML at build time
- API Routes — backend endpoints inside the same project
- Image optimization — automatic WebP, lazy loading
- Font optimization — zero layout shift
- Built-in CSS Modules, Sass, Tailwind support
- TypeScript out of the box
- Edge and Node.js runtime support

When to use Next.js:

- SEO matters — SSR sends pre-rendered HTML to crawlers
- E-commerce, blogs, marketing sites — SSG for speed
- Full-stack apps — API routes + React UI in one repo
- Dashboard apps — hybrid rendering per page

Next.js vs Create React App:

- CRA: SPA, client-only, no SSR, deprecated
- Next.js: hybrid, SSR/SSG, routing, API, production-ready

```
// Create a Next.js app
// npx create-next-app@latest my-app
// npx create-next-app@latest my-app --typescript --tailwind --app

// pages/index.tsx (Pages Router)
export default function Home() {
  return <h1>Hello Next.js!</h1>
}

// app/page.tsx (App Router — Next.js 13+)
export default function HomePage() {
  return <h1>Hello App Router!</h1>
}

// Server component — fetches data WITHOUT useEffect/useState
export default async function ProductsPage() {
  const products = await fetch('https://api.example.com/products')
    .then(r => r.json())
  return (
    <ul>
      {products.map(p => <li key={p.id}>{p.name}</li>)}
```

```
    </ul>
```

```
  )
}
// ↑ This runs on the SERVER — zero client JS for data fetching

// next.config.js
/** @type {import('next').NextConfig} */
const nextConfig = {
  images: { domains: ['cdn.example.com'] },
  experimental: { serverActions: true }
}
module.exports = nextConfig
```

■ Interview Tip:
Next.js is the default choice for React in 2024+. Even Vercel's own React team recommends starting with Next.js. The main reason: SSR and SSG dramatically improve SEO and initial page load — CRA apps score poorly on Core Web Vitals compared to Next.js apps.

Q2 Pages Router vs App Router — what's the difference?

■ Answer

Next.js has two routing systems: the original Pages Router and the new App Router (Next.js 13+).
The App Router is the future — it uses React Server Components and is recommended for new projects.

Pages Router (legacy, still supported):

- Files in /pages directory
- Each file = one route automatically
- getStaticProps / getServerSideProps for data fetching
- _app.tsx for global layout
- _document.tsx for HTML shell
- API routes in pages/api/

App Router (Next.js 13+, recommended):

- Files in /app directory
- Uses React Server Components by default
- Layout files (layout.tsx) for shared UI
- loading.tsx, error.tsx, not-found.tsx per segment
- Route Groups — (group) folder name
- Server Actions for form handling
- Colocate components, tests, styles with routes

Key differences:

- Data fetching: getServerSideProps → async components
- Client state: pages use useState freely, app uses 'use client'
- Layouts: _app.tsx → layout.tsx (nested!)
- API: pages/api/ → app/api/route.ts

```
// ■■ PAGES ROUTER structure ■■
// pages/
//   index.tsx          → /
//   about.tsx          → /about
//   blog/[slug].tsx    → /blog/:slug
//   api/users.ts       → /api/users
//   _app.tsx           → wraps all pages
//   _document.tsx      → HTML document

// pages/blog/[slug].tsx
export async function getStaticProps({ params }) {
  const post = await fetchPost(params.slug)
  return { props: { post } }
}
export default function BlogPost({ post }) {
  return <article>{post.title}</article>
}

// ■■ APP ROUTER structure ■■
```

```
// app/
```

```
21 //   layout.tsx          → root layout (wraps everything)
22 //   page.tsx            → /
23 //   about/page.tsx      → /about
24 //   blog/[slug]/
25 //     page.tsx          → /blog/:slug
26 //     loading.tsx       → loading UI
27 //     error.tsx         → error UI
28 //   api/users/route.ts → /api/users
29
30 // app/blog/[slug]/page.tsx — Server Component, async by default
31 export default async function BlogPost({ params }) {
32   const post = await fetchPost(params.slug)  // direct async!
33   return <article>{post.title}</article>
34 }
35
36 // Can use BOTH routers in the same project during migration
37 // pages/ and app/ can coexist — app/ takes priority
```

■ Interview Tip:
New projects should use the App Router. Existing Pages Router projects can migrate gradually — both can coexist. The App Router's biggest win is React Server Components: zero client JS for components that don't need interactivity.

Q3 What is file-based routing in Next.js?

■ Answer

Next.js creates routes automatically based on the file system structure.
No router configuration needed — the file path IS the URL path.

App Router routing rules:

- app/page.tsx → /
- app/about/page.tsx → /about
- app/blog/[slug]/page.tsx → /blog/:slug (dynamic)
- app/shop/[...slug]/page.tsx → /shop/a/b/c (catch-all)
- app/shop/[[...slug]]/page.tsx → /shop and /shop/a/b (optional catch-all)
- app/(marketing)/page.tsx → / (route group, no URL segment)
- app/@modal/page.tsx → parallel route slot

Special files in App Router:

- page.tsx — the route UI
- layout.tsx — shared wrapper (persists on navigation!)
- loading.tsx — Suspense wrapper with skeleton
- error.tsx — error boundary
- not-found.tsx — 404 UI
- route.ts — API route handler
- template.tsx — layout that re-mounts on navigation

Dynamic segments:

- [id] — single dynamic segment
- [...slug] — catch-all (must have at least one segment)
- [[...slug]] — optional catch-all (including empty)

```
// app/
// ■■■ layout.tsx           → / (root layout)
// ■■■ page.tsx             → /
// ■■■ about/
// ■   ■■■ page.tsx         → /about
// ■■■ blog/
// ■   ■■■ page.tsx         → /blog
// ■   ■■■ [slug]/
// ■       ■■■ page.tsx     → /blog/hello-world
// ■       ■■■ loading.tsx  → loading state for /blog/:slug
// ■■■ shop/
// ■   ■■■ [...category]/
// ■       ■■■ page.tsx     → /shop/electronics/phones/iphone
// ■■■ (auth)/              → route group — no URL impact
// ■   ■■■ login/page.tsx   → /login
// ■   ■■■ register/page.tsx → /register
// ■■■ api/
//     ■■■ users/
//         ■■■ route.ts     → /api/users
```

```
```

```tsx
// app/blog/[slug]/page.tsx
interface Props {
  params: { slug: string }
  searchParams: { [key: string]: string | string[] }
}

export default async function BlogPost({ params, searchParams }: Props) {
  const { slug } = params  // 'hello-world'
  const page = searchParams.page  // ?page=2
  const post = await getPost(slug)
  return <article>{post.content}</article>
}

// app/shop/[...category]/page.tsx
export default function ShopPage({ params }: { params: { category: string[] } }) {
  // /shop/electronics/phones → params.category = ['electronics', 'phones']
  return <div>{params.category.join(' > ')}</div>
}
```

■ Interview Tip:
The route group pattern (folder in parentheses) is one of Next.js's most useful features. Use it to share layouts without affecting URLs: (dashboard)/settings and (dashboard)/profile both use the dashboard layout, but the URL is just /settings and /profile.

Q4 Layouts and Templates in Next.js App Router?

■ Answer

Layouts are shared UI that wraps child routes and PERSISTS across navigation.
Templates are similar but RE-MOUNT on every navigation (losing state).

layout.tsx:

- Wraps all child routes in the same directory
- Component does NOT unmount on navigation
- State is preserved when navigating between children
- Must accept children prop
- Root layout (app/layout.tsx) MUST include html and body tags
- Can be nested — child layouts wrap their subtree

template.tsx:

- Same as layout but creates a NEW instance on navigation
- Component DOES unmount and remount
- State is LOST on navigation
- Use for: page transition animations, resetting per-page state
- useEffect runs on every navigation

Layout hierarchy:

- app/layout.tsx → app/dashboard/layout.tsx → app/dashboard/settings/page.tsx
- Each layout wraps the next level
- Inner layouts can use outer layout's context

```
// app/layout.tsx — ROOT layout (required!)
import type { Metadata } from 'next'
import { Inter } from 'next/font/google'
import './globals.css'

const inter = Inter({ subsets: ['latin'] })

export const metadata: Metadata = {
  title: { template: '%s | MyApp', default: 'MyApp' },
  description: 'My awesome app'
}

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang='en'>
      <body className={inter.className}>
        <nav>Global Navigation</nav>
        {children}  {/* page content renders here */}
        <footer>Footer</footer>
      </body>
    </html>
  )
```

```
}
```

```
// app/dashboard/layout.tsx — NESTED layout
export default function DashboardLayout({ children }: { children: React.ReactNode }) {
  return (
    <div className='flex'>
      <aside>
        <nav>
          <a href='/dashboard'>Overview</a>
          <a href='/dashboard/settings'>Settings</a>
        </nav>
      </aside>
      <main className='flex-1'>{children}</main>
    </div>
  )
}
// Renders as: RootLayout → DashboardLayout → page

// app/dashboard/template.tsx — template (remounts!)
// Use for page entry animations
'use client'
import { motion } from 'framer-motion'
export default function Template({ children }: { children: React.ReactNode }) {
  return (
    <motion.div
      initial={{ opacity: 0 }}
      animate={{ opacity: 1 }}
    >
      {children}
    </motion.div>
  )
}
```

■ Interview Tip:
The biggest layout mistake: adding <html> and <body> tags in nested layouts. Only the ROOT layout (app/layout.tsx) should have them. Also remember: layouts don't re-render on navigation to child routes — great for performance, but means you can't read searchParams in layouts.

Q5 Link Component vs <a> tag in Next.js?

■ Answer

Next.js provides a <Link> component that enables client-side navigation with prefetching. Using a plain <a> tag causes a full page reload — losing React state and slow.

<Link> component:

- Client-side navigation — no full page reload
- Automatically prefetches linked pages in viewport
- Preserves React state across navigation
- Scroll restoration built-in
- Works with App Router and Pages Router

Link props:

- href (required) — path or object
- replace — replace history instead of push
- scroll — false to disable scroll-to-top
- prefetch — false to disable prefetching
- className — styling

useRouter() for programmatic navigation:

- router.push('/path') — navigate programmatically
- router.replace('/path') — replace history
- router.back() — go back
- router.refresh() — refresh server components

When to use <a> vs <Link>:

- <Link> — internal Next.js pages
- <a href> — external URLs, file downloads

```
import Link from 'next/link'
import { useRouter } from 'next/navigation'  // App Router
// import { useRouter } from 'next/router'  // Pages Router

// Basic navigation
function Nav() {
  return (
    <nav>
      {/* ■ Internal links — use Link */}
      <Link href='/'>Home</Link>
      <Link href='/about'>About</Link>
      <Link href='/blog/hello-world'>Blog Post</Link>

      {/* Dynamic routes */}
      <Link href={'/products/${product.id}'}>View Product</Link>

      {/* With query params */}
      <Link href={{ pathname: '/search', query: { q: 'nextjs', page: 2 } }}>
```

```
        Search Results
```

```
      </Link>

      {/* Replace history — no back button */}
      <Link href='/login' replace>Login</Link>

      {/* Disable scroll to top */}
      <Link href='/comments#section' scroll={false}>Jump to Comments</Link>

      {/* Disable prefetch */}
      <Link href='/rare-page' prefetch={false}>Rarely visited</Link>

      {/* ■ External links — use <a> */}
      <a href='https://github.com' target='_blank' rel='noopener noreferrer'>
        GitHub
      </a>
    </nav>
  )
}

// Programmatic navigation — must be in 'use client' component
'use client'
function LoginButton() {
  const router = useRouter()

  async function handleLogin() {
    await loginUser()
    router.push('/dashboard')   // navigate after login
    // router.replace('/dashboard')  // no back button
    // router.refresh()  // refresh server data
  }
  return <button onClick={handleLogin}>Login</button>
}
```

■ Interview Tip:
<Link> prefetches pages when they enter the viewport in production. This means by the time a user clicks a link, the destination page is already loaded — giving near-instant navigation. Disable prefetch={false} only for rarely visited pages to save bandwidth.

Q6 Image Optimization with next/image?

■ Answer

The next/image component automatically optimizes images for performance.
It handles: lazy loading, WebP conversion, responsive sizes, and prevents layout shift.

What next/image does automatically:

- Lazy loads images (only loads when in viewport)
- Converts to WebP/AVIF for supported browsers
- Serves correct size for device (srcset)
- Prevents Cumulative Layout Shift (CLS) via width/height
- CDN delivery via Vercel or custom loader

Required props:

- src — URL string or imported image
- alt — accessibility text
- width + height OR fill — defines space

Important props:

- priority — load eagerly (LCP images! Above the fold)
- fill — fills parent container (parent needs position:relative)
- sizes — responsive size hints for srcset
- quality — compression 1-100 (default 75)
- placeholder='blur' — blur-up effect while loading
- loading='eager' — disable lazy loading

External images:

- Must whitelist domains in next.config.js
- Use remotePatterns for hostname patterns

```
import Image from 'next/image'
import heroImage from '@/public/hero.jpg'  // static import

// ■■ STATIC import (recommended — auto width/height + blur)
function Hero() {
  return (
    <Image
      src={heroImage}
      alt='Hero image'
      priority          // LCP image — load eagerly, no lazy!
      placeholder='blur' // blurDataURL auto-generated
      className='w-full object-cover'
    />
  )
}

// ■■ REMOTE image (URL string)
function Avatar({ user }) {
```

```
  return (
```

```
20      <Image
21        src={user.avatarUrl}   // 'https://cdn.example.com/avatar.jpg'
22        alt={user.name}
23        width={48}
24        height={48}
25        className='rounded-full'
26      />
27    )
28  }
29
30  // ■■ FILL mode — image fills parent
31  function ProductImage({ src }) {
32    return (
33      <div className='relative aspect-square w-full'>
34        <Image
35          src={src}
36          alt='Product'
37          fill
38          sizes='(max-width: 768px) 100vw, 50vw'  // hint for srcset
39          className='object-cover'
40        />
41      </div>
42    )
43  }
44
45  // next.config.js — allow external domains
46  module.exports = {
47    images: {
48      remotePatterns: [
49        { protocol: 'https', hostname: 'cdn.example.com' },
50        { protocol: 'https', hostname: '**.cloudinary.com' },
51      ]
52    }
53  }
```

■ Interview Tip:
Always add priority to above-the-fold images (your hero image, logo). These are your Largest Contentful Paint (LCP) elements — lazy loading them hurts Core Web Vitals scores. Also always provide meaningful sizes prop for fill images so Next.js generates optimal srcsets.

Q7 Static Generation vs SSR vs SSG vs ISR — when to use each?

■ Answer

Next.js offers four rendering strategies — choosing the right one per page is critical.

CSR (Client-Side Rendering):

- Rendering happens in browser after JS loads
- Empty HTML shell initially → bad for SEO
- Use for: dashboards behind auth, user-specific data
- How: 'use client' + useState/useEffect or React Query

SSG (Static Site Generation):

- HTML built at BUILD TIME — fastest possible delivery
- Served from CDN — no server needed per request
- Use for: blog posts, marketing pages, docs, product listings
- App Router: async Server Component with no dynamic functions

SSR (Server-Side Rendering):

- HTML built on each REQUEST
- Fresh data always — good for personalized content
- Slower than SSG — requires server per request
- Use for: user dashboard, cart, search results
- App Router: use cookies(), headers(), or searchParams

ISR (Incremental Static Regeneration):

- Static page with time-based revalidation
- Best of both: CDN speed + fresh data
- Use for: product pages, news articles, prices
- App Router: fetch with revalidate option

Decision guide:

- Public, rarely changes → SSG
- Public, changes regularly → ISR
- Private/personalized → SSR or CSR

```
// ■■ SSG — static, built at build time
// app/blog/[slug]/page.tsx
export async function generateStaticParams() {
  const posts = await getPosts()
  return posts.map(post => ({ slug: post.slug }))
}

export default async function BlogPost({ params }) {
  const post = await getPost(params.slug)  // runs at BUILD TIME
  return <article>{post.content}</article>
}
// No dynamic functions = Next.js auto-statically generates this

```

```
// ■■ ISR — static + time-based revalidation
```

```
// app/products/[id]/page.tsx
export default async function ProductPage({ params }) {
  const product = await fetch('/api/products/${params.id}', {
    next: { revalidate: 3600 }  // re-generate every 1 hour
  }).then(r => r.json())
  return <div>{product.name}: ${product.price}</div>
}

// ■■ SSR — rendered on every request
// app/dashboard/page.tsx
import { cookies } from 'next/headers'

export default async function Dashboard() {
  const token = cookies().get('token')?.value
  const user = await getUser(token)  // runs on EVERY request
  return <div>Welcome, {user.name}</div>
}
// cookies() makes this SSR automatically!

// ■■ CSR — client side only
'use client'
export default function UserProfile() {
  const [profile, setProfile] = useState(null)
  useEffect(() => { fetchProfile().then(setProfile) }, [])
  return profile ? <div>{profile.name}</div> : <Loading />
}
```

■ Interview Tip:
The rendering decision should be per-page, not per-app. A Next.js app can have SSG for the homepage, ISR for product pages, SSR for the cart, and CSR for the real-time dashboard — all in one project. This hybrid approach is Next.js's biggest advantage.

Q8 next.config.js — important configuration options?

■ Answer

next.config.js is the configuration file for your Next.js application.
It controls build behavior, environment variables, redirects, rewrites, and more.

Key configuration options:

- images — remote patterns, formats, sizes
- env — public environment variables
- redirects() — redirect rules
- rewrites() — proxy/mask URLs
- headers() — custom HTTP headers
- experimental — experimental features
- output — 'standalone' for Docker, 'export' for static
- basePath — deploy under a subpath
- trailingSlash — add/remove trailing slashes
- compiler — SWC options
- webpack — customize webpack config

Environment variables in next.config.js:

- NEXT_PUBLIC_ prefix — exposed to client AND server
- No prefix — server-only (not in client bundle)
- Use .env.local for local overrides

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  // Image optimization
  images: {
    remotePatterns: [
      { protocol: 'https', hostname: 'images.unsplash.com' },
      { protocol: 'https', hostname: '**.amazonaws.com' }
    ],
    formats: ['image/avif', 'image/webp'],
    minimumCacheTTL: 60
  },

  // Redirects (301 by default)
  async redirects() {
    return [
      { source: '/old-blog/:slug', destination: '/blog/:slug', permanent: true },
      { source: '/docs', destination: 'https://docs.example.com', permanent: false }
    ]
  },

  // Rewrites — proxy API without exposing URL
  async rewrites() {
    return [
      { source: '/api/:path*', destination: 'https://api.backend.com/:path*' }
```

```
    ]
```

```
  },

  // Custom HTTP headers
  async headers() {
    return [
      {
        source: '/(.*)',
        headers: [
          { key: 'X-Frame-Options', value: 'DENY' },
          { key: 'X-Content-Type-Options', value: 'nosniff' }
        ]
      }
    ]
  },

  // Build output
  output: 'standalone',  // for Docker — single self-contained output

  // Experimental features
  experimental: {
    serverActions: { allowedOrigins: ['localhost:3000'] },
    ppr: true,  // Partial Pre-Rendering (Next.js 14)
  }
}

module.exports = nextConfig

// .env.local
// DATABASE_URL=mongodb://...    ← server only
// NEXT_PUBLIC_API_URL=https://api.example.com  ← client + server
```

■ Interview Tip:
Use rewrites() to avoid exposing your backend API URL to clients. Instead of calling https://api.backend.com directly from the browser, clients call /api/* and Next.js proxies the request. This hides your backend architecture and enables adding auth headers server-side.

Q9 What are React Server Components (RSC)? How do they work?

■ Answer

React Server Components run exclusively on the server and send HTML (not JS) to the client.
This eliminates client-side JavaScript for components that don't need interactivity.

Server Components (default in App Router):

- Run on the server — access databases, file system, env vars
- NEVER hydrated on client — zero client JS bundle impact
- Can be async — await data directly
- Cannot use: useState, useEffect, event handlers, browser APIs
- Cannot use: Context (only in Client Components)

Client Components ('use client'):

- Run on BOTH server (initial render) AND client (hydration)
- Can use: useState, useEffect, event handlers, browser APIs
- Add to client JS bundle — each adds to page weight
- Use for: interactivity, user events, real-time updates

The boundary:

- 'use client' creates a client component boundary
- All imports below the boundary become client-side too
- Server Component CAN render Client Component (pass as children)
- Client Component CANNOT render Server Component directly

Benefits of RSC:

- Smaller bundle — server logic stays on server
- Secure — secrets never reach client
- Faster — no client-side data fetching waterfalls

```
// ■■ SERVER COMPONENT (default — no directive needed) ■■
// app/dashboard/page.tsx
import { db } from '@/lib/db'  // Direct DB access! Never exposed to client

export default async function DashboardPage() {
  // Direct database query — no API route needed
  const stats = await db.query('SELECT count(*) FROM orders')
  const user = await db.users.findById(getCurrentUserId())

  return (
    <div>
      <h1>Welcome, {user.name}</h1>
      {/* InteractiveChart is a Client Component */}
      <InteractiveChart data={stats} />
    </div>
  )
}
// Zero client JS for this component!
```

```
// ■■ CLIENT COMPONENT ■■
// components/InteractiveChart.tsx
'use client'
import { useState } from 'react'

export default function InteractiveChart({ data }) {
  const [view, setView] = useState('weekly')

  return (
    <div>
      <button onClick={() => setView('weekly')}>Weekly</button>
      <button onClick={() => setView('monthly')}>Monthly</button>
      <Chart data={data} view={view} />
    </div>
  )
}

// ■ Pass Server Component as children to Client Component
// This works! children renders on server, wrapper on client
function ClientWrapper({ children }) {  // 'use client'
  return <div className='animated'>{children}</div>
}
// In Server Component:
// <ClientWrapper><ServerComponent /></ClientWrapper>  ■

// ■ Cannot import Server Component inside Client Component
// 'use client'
// import ServerComponent from './Server'  // Error!
```

■ Interview Tip:
The biggest RSC mindset shift: stop thinking 'where does my data fetching happen' and start thinking 'does this component need interactivity?' If no — keep it a Server Component. Access your database directly. The component's output (HTML) goes to the client, not the code.

Q10 'use client' vs 'use server' directives?

■ Answer

These directives mark where a module runs — they define the server/client boundary.

'use client':

- Marks a module (and all its imports) as client-side
- Component runs on server for initial render THEN hydrates on client
- Required for: useState, useEffect, event handlers, browser APIs
- Creates a 'client boundary' — everything below runs on client
- Must be the FIRST line of the file

'use server':

- Marks a function as a Server Action
- Function runs on the server even when called from client
- Use for: form submissions, mutations, database writes
- Can be in a separate file: 'use server' at top of file
- Or inline in Server Components for one-off actions
- Automatically creates an API endpoint under the hood

When to add 'use client':

- useState, useReducer, useContext
- useEffect, useRef, useCallback
- onClick, onChange, any event handler
- window, document, localStorage access
- Third-party libraries that use hooks

```
// ■■ 'use client' ■■
'use client'  // MUST be first line

import { useState, useEffect } from 'react'

export default function Counter() {
  const [count, setCount] = useState(0)
  return <button onClick={() => setCount(c => c + 1)}>{count}</button>
}

// ■■ 'use server' — Server Actions ■■

// actions.ts — entire file is server actions
'use server'

import { db } from '@/lib/db'
import { revalidatePath } from 'next/cache'

export async function createPost(formData: FormData) {
  const title = formData.get('title') as string
  const content = formData.get('content') as string
```

```
  // Direct DB write — runs on server!
  await db.posts.create({ data: { title, content } })

  revalidatePath('/blog')  // refresh the blog page cache
}

export async function deletePost(id: string) {
  await db.posts.delete({ where: { id } })
  revalidatePath('/blog')
}

// Using in a form (no JavaScript required!)
import { createPost } from './actions'

export default function CreatePostForm() {
  return (
    <form action={createPost}>  {/* Server Action! */}
      <input name='title' placeholder='Title' />
      <textarea name='content' />
      <button type='submit'>Create Post</button>
    </form>
  )
}

// 'use server' inline in Server Component
export default function Page() {
  async function handleSubmit(formData: FormData) {
    'use server'  // inline server action
    await createRecord(formData)
  }
  return <form action={handleSubmit}>...</form>
}
```

■ Interview Tip:
Server Actions are a paradigm shift — they let you write server-side mutation logic (form handling, DB writes) directly in your React component file without creating separate API routes. The 'use server' directive creates a secure RPC-like call under the hood.

Q11 Layouts and Nested Layouts — advanced patterns?

■ Answer

Nested layouts allow different sections of your app to have independent persistent UI.
Each route segment can have its own layout that wraps its children.

Layout nesting order:

- app/layout.tsx → wraps everything
- app/dashboard/layout.tsx → wraps /dashboard/*
- app/dashboard/settings/layout.tsx → wraps /dashboard/settings/*

Layouts receive:

- children — the inner page/layout
- params — dynamic route segments (in dynamic routes)

Accessing route info in layouts:

- usePathname() in Client Component
- useSelectedLayoutSegment() — active child segment
- Cannot use searchParams — layouts don't re-render for search param changes

Route groups for layout sharing:

- (group) folders group routes under one layout
- Without affecting the URL

Parallel routes in layouts:

- @slot convention for rendering multiple pages simultaneously
- Dashboard with sidebar + main content independently routed

```
// app/layout.tsx (root)
export default function RootLayout({ children }) {
  return <html><body><TopBar />{children}</body></html>
}

// app/(dashboard)/layout.tsx  ← route group, URL unaffected
'use client'
import { useSelectedLayoutSegment } from 'next/navigation'

export default function DashboardLayout({ children }) {
  const segment = useSelectedLayoutSegment()  // 'analytics' | 'settings' | null

  return (
    <div className='flex h-screen'>
      <Sidebar activeSegment={segment} />
      <main className='flex-1 overflow-auto'>{children}</main>
    </div>
  )
}

// app/(dashboard)/analytics/layout.tsx — NESTED layout
```

```
export default function AnalyticsLayout({ children }) {
```

```
  return (
    <div>
      <AnalyticsNav />  {/* Date range picker, filters, etc. */}
      {children}
    </div>
  )
}

// Parallel slots — @folder convention
// app/
//   @sidebar/page.tsx     → sidebar content
//   @main/page.tsx        → main content
//   layout.tsx            → receives both as props

// app/layout.tsx with parallel routes
export default function Layout({
  children,
  sidebar,  // @sidebar slot
  modal,    // @modal slot
}: {
  children: React.ReactNode
  sidebar: React.ReactNode
  modal: React.ReactNode
}) {
  return (
    <div>
      {sidebar}
      {children}
      {modal}
    </div>
  )
}
```

■ Interview Tip:
The useSelectedLayoutSegment hook is perfect for building active nav links in sidebar layouts — it tells you which child segment is active without needing usePathname comparison. Use it in the dashboard layout's sidebar to highlight the current page.

Q12 loading.tsx and error.tsx — automatic UI states?

■ Answer

loading.tsx and error.tsx automatically wrap route segments in Suspense boundaries and error boundaries. They provide zero-config loading and error states per route.

loading.tsx:

- Automatically wrapped in React Suspense
- Shows while the page's async data loads
- Shown IMMEDIATELY — doesn't wait for JS
- The layout renders, loading.tsx shown, then page replaces it
- Use for: skeleton screens, spinners, progress bars

error.tsx:

- Must be a Client Component ('use client')
- Wraps the route in a React Error Boundary
- Receives: error (Error object) and reset (retry function)
- Does NOT catch errors in: layout, not-found, or the error component itself

not-found.tsx:

- Shown when notFound() is called or 404
- Place in app/ for global 404, or in segments for scoped 404

global-error.tsx:

- Catches errors in root layout
- Must include html + body tags
- Replaces the layout entirely when error occurs

```
// app/dashboard/loading.tsx
export default function DashboardLoading() {
  return (
    <div className='animate-pulse space-y-4'>
      <div className='h-8 w-48 bg-gray-200 rounded' />
      <div className='grid grid-cols-3 gap-4'>
        {[1,2,3].map(i => (
          <div key={i} className='h-32 bg-gray-200 rounded' />
        ))}
      </div>
    </div>
  )
}

// app/dashboard/error.tsx
'use client'

export default function DashboardError({
  error,
  reset,
```

```
}: {
```

```
  error: Error & { digest?: string }
  reset: () => void
}) {
  return (
    <div className='text-center py-10'>
      <h2 className='text-xl font-bold text-red-600'>Something went wrong!</h2>
      <p className='text-gray-600'>{error.message}</p>
      <button
        onClick={reset}  // re-render the segment
        className='mt-4 px-4 py-2 bg-blue-500 text-white rounded'
      >
        Try Again
      </button>
    </div>
  )
}

// app/dashboard/not-found.tsx
import Link from 'next/link'

export default function NotFound() {
  return (
    <div>
      <h2>Dashboard Not Found</h2>
      <Link href='/dashboard'>Return to Dashboard</Link>
    </div>
  )
}

// Trigger not-found from Server Component
import { notFound } from 'next/navigation'

export default async function Page({ params }) {
  const item = await getItem(params.id)
  if (!item) notFound()  // triggers not-found.tsx
  return <div>{item.name}</div>
}
```

■ Interview Tip:
loading.tsx is powered by Suspense streaming — the shell (layout + loading skeleton) is sent immediately, then the actual page content streams in as it resolves. This gives users instant visual feedback even before the data arrives, dramatically improving perceived performance.

Q13 Route Groups — (folder) pattern?

■ Answer

Route groups use parentheses in folder names to organize routes WITHOUT creating URL segments.

What route groups enable:

- Share a layout between specific routes without affecting URL
- Organize routes by feature, team, or purpose
- Multiple root layouts (different html/body for marketing vs app)
- Opt-in to different error/loading boundaries

Naming:

- (group-name) — parentheses mean 'no URL segment'
- (marketing)/about/page.tsx → /about (NOT /marketing/about)

Common patterns:

- (auth) — login, register, forgot-password share auth layout
- (dashboard) — all authenticated pages share sidebar layout
- (marketing) — public pages share marketing layout
- (admin) — admin pages share admin layout + middleware

```
// Organizing by layout groups
// app/
// ■■■ (marketing)/         ← route group, no URL segment
// ■   ■■■ layout.tsx       ← marketing layout (header, footer, etc.)
// ■   ■■■ page.tsx         → /
// ■   ■■■ about/page.tsx   → /about
// ■   ■■■ pricing/page.tsx → /pricing
// ■■■ (auth)/              ← auth group
// ■   ■■■ layout.tsx       ← centered card layout
// ■   ■■■ login/page.tsx   → /login
// ■   ■■■ register/page.tsx → /register
// ■■■ (app)/               ← authenticated app
// ■   ■■■ layout.tsx       ← sidebar layout
// ■   ■■■ dashboard/page.tsx → /dashboard
// ■   ■■■ settings/page.tsx  → /settings

// app/(marketing)/layout.tsx
export default function MarketingLayout({ children }) {
  return (
    <>
      <MarketingHeader />
      {children}
      <Footer />
    </>
  )
}

// app/(auth)/layout.tsx
```

```
export default function AuthLayout({ children }) {
```

```
  return (
    <div className='min-h-screen flex items-center justify-center bg-gray-50'>
      <div className='w-full max-w-md'>{children}</div>
    </div>
  )
}

// MULTIPLE ROOT LAYOUTS — different html/body per group
// app/(marketing)/layout.tsx — returns <html>
// app/(app)/layout.tsx — returns <html>
// (omit root app/layout.tsx)
// ↑ Each group has a completely separate HTML shell
```

■ Interview Tip:
Multiple root layouts is a powerful pattern for apps with completely different shells — a marketing site with a classic header/footer and a SaaS app with a sidebar don't need to share a root layout. Just omit app/layout.tsx and give each route group its own root layout.

Q14 Parallel Routes and Intercepting Routes?

■ Answer

Parallel routes render multiple pages simultaneously in the same layout — like a modal overlay with its own URL. Intercepting routes capture a navigation and show different content inline.

Parallel Routes (@slot):

- Folder named @slotName
- Layout receives slot as a prop
- Each slot can have its own loading/error states
- Slots can be conditionally rendered
- Use for: dashboard with independent sections, split views

Intercepting Routes ((.)path):

- (.) — same level
- (..) — one level up
- (...) — root level
- When you click a link, Next.js shows the intercepting route
- On direct URL visit or refresh, shows the real route
- Use for: photo gallery modal, login modal, quick preview

Combining both:

- @modal + intercepting route = URL-aware modal
- Social media photo viewing pattern

```
// ■■ PARALLEL ROUTES ■■
// app/
// ■■■ @analytics/
// ■   ■■■ page.tsx     ← analytics panel
// ■■■ @team/
// ■   ■■■ page.tsx     ← team panel
// ■■■ layout.tsx       ← receives both

// app/layout.tsx
export default function Layout({
  children,
  analytics,  // @analytics slot
  team,       // @team slot
}) {
  return (
    <div className='grid grid-cols-3 gap-4'>
      <div className='col-span-2'>{children}</div>
      <div>
        {analytics}
        {team}
      </div>
    </div>
  )
}
```

```
// ■■ INTERCEPTING ROUTES + MODAL pattern ■■
// app/
// ■■■ @modal/
// ■   ■■■ (.)photo/[id]/
// ■       ■■■ page.tsx  ← modal view (intercepting)
// ■■■ photo/
// ■   ■■■ [id]/
// ■       ■■■ page.tsx  ← full page view
// ■■■ layout.tsx

// layout.tsx
export default function Layout({ children, modal }) {
  return (
    <>
      {children}
      {modal}  {/* renders modal slot if intercepting */}
    </>
  )
}

// @modal/(.)photo/[id]/page.tsx
import Modal from '@/components/Modal'
export default async function PhotoModal({ params }) {
  const photo = await getPhoto(params.id)
  return (
    <Modal>  {/* overlay — click outside = back */}
      <img src={photo.url} alt={photo.title} />
    </Modal>
  )
}
// Click photo in gallery → modal shows (URL updates to /photo/123)
// Refresh /photo/123 → full page view
```

■ Interview Tip:
The intercepting routes + parallel routes combination creates the Instagram/Twitter photo modal UX: clicking a photo opens a modal with the photo's URL, sharing that URL opens the full photo page, and pressing back closes the modal and returns to the feed.

Q15 generateStaticParams() — dynamic static generation?

■ Answer

generateStaticParams() pre-defines which dynamic route values to statically generate at build time. It replaces getStaticPaths() from the Pages Router.

How it works:

- Export from a dynamic route's page.tsx or layout.tsx
- Returns an array of param objects to pre-generate
- At build time, Next.js generates one static page per item
- Requests for non-listed params: 404 or dynamic fallback

dynamicParams:

- true (default) — non-listed params are server-rendered on demand
- false — non-listed params → 404

Optimizations:

- Can fetch data in generateStaticParams — cached across all segment renders
- Nested dynamic routes: all params from parent are available
- Combine with revalidate for ISR-like behavior

When to use:

- Blog posts with known slugs
- Product pages from a catalog
- User profiles (popular ones)
- Docs pages from a CMS

```
// app/blog/[slug]/page.tsx
import { getAllPosts, getPost } from '@/lib/blog'

// 1. Tell Next.js which slugs to pre-generate
export async function generateStaticParams() {
  const posts = await getAllPosts()  // fetch all posts
  return posts.map(post => ({
    slug: post.slug  // returns [{ slug: 'hello' }, { slug: 'world' }]
  }))
}

// 2. The page renders statically for each slug
export default async function BlogPost({ params }: { params: { slug: string } }) {
  const post = await getPost(params.slug)
  return <article>{post.content}</article>
}

// 3. Control non-pre-generated params
export const dynamicParams = true   // render on-demand if not pre-generated
// export const dynamicParams = false  // return 404 for unlisted slugs

// Nested dynamic routes
```

```
// app/[category]/[slug]/page.tsx
```

```
24 export async function generateStaticParams() {
25   const posts = await getAllPosts()
26   return posts.map(post => ({
27     category: post.category,
28     slug: post.slug
29   }))
30   // [{ category: 'tech', slug: 'nextjs-13' }, ...]
31 }
32
33 // Generate metadata for each static page
34 export async function generateMetadata({ params }) {
35   const post = await getPost(params.slug)
36   return {
37     title: post.title,
38     description: post.excerpt,
39     openGraph: {
40       title: post.title,
41       images: [post.ogImage]
42     }
43   }
44 }
```

■ Interview Tip:
generateStaticParams runs once at build time and the results are cached. If you fetch data inside it, that fetch is automatically deduplicated with fetches inside the page component — so you don't hit your API twice for the same post data during static generation.

Q16 Metadata and Open Graph in Next.js?

■ Answer

Next.js has a built-in Metadata API for managing <head> elements, SEO, and social sharing cards.

Static metadata:

- Export a metadata object from page.tsx or layout.tsx
- Handles: title, description, keywords, robots, etc.

Dynamic metadata:

- Export generateMetadata() function
- Receives params and searchParams
- Can fetch data for dynamic SEO

Title template:

- { template: '%s | MyApp', default: 'MyApp' }
- Child pages fill in %s, root layout provides fallback

Open Graph:

- metadata.openGraph — Facebook, LinkedIn sharing
- metadata.twitter — Twitter cards

File-based metadata:

- favicon.ico in app/ folder — auto-detected
- opengraph-image.png — auto OG image
- opengraph-image.tsx — dynamically generated OG image
- sitemap.ts — auto-generated sitemap
- robots.ts — auto-generated robots.txt

```
// app/layout.tsx — base metadata
import type { Metadata } from 'next'

export const metadata: Metadata = {
  title: {
    template: '%s | MyApp',
    default: 'MyApp — The Best App'
  },
  description: 'MyApp helps you do amazing things',
  keywords: ['next.js', 'react', 'app'],
  authors: [{ name: 'Kaushal' }],
  metadataBase: new URL('https://myapp.com'),  // for absolute OG URLs
  openGraph: {
    type: 'website',
    locale: 'en_US',
    url: 'https://myapp.com',
    siteName: 'MyApp',
    images: [{ url: '/og-default.png', width: 1200, height: 630 }]
  },
  twitter: { card: 'summary_large_image', creator: '@kaushal' },
```

```
  robots: { index: true, follow: true }
```

```
}

// app/blog/[slug]/page.tsx — dynamic metadata
export async function generateMetadata(
  { params }: { params: { slug: string } }
): Promise<Metadata> {
  const post = await getPost(params.slug)
  if (!post) return { title: 'Post Not Found' }

  return {
    title: post.title,  // '%s | MyApp' → 'Post Title | MyApp'
    description: post.excerpt,
    openGraph: {
      title: post.title,
      description: post.excerpt,
      url: 'https://myapp.com/blog/${post.slug}',
      images: [{ url: post.coverImage, width: 1200, height: 630 }]
    }
  }
}

// app/opengraph-image.tsx — dynamically generated OG image
import { ImageResponse } from 'next/og'

export default function OGImage() {
  return new ImageResponse(
    <div style={{ display: 'flex', background: '#1a1a2e' }}>
      <h1 style={{ color: 'white' }}>MyApp</h1>
    </div>,
    { width: 1200, height: 630 }
  )
}

// app/sitemap.ts
export default async function sitemap() {
  const posts = await getPosts()
  return posts.map(post => ({
    url: 'https://myapp.com/blog/${post.slug}',
    lastModified: post.updatedAt
  }))
}
```

■ Interview Tip:
Set metadataBase in your root layout — without it, relative URLs in openGraph.images are resolved incorrectly in production. Also, the opengraph-image.tsx approach is gold: Next.js generates OG images on-demand using @vercel/og (Edge Runtime), no separate screenshot service needed.

Q17 getStaticProps and getServerSideProps — Pages Router data fetching?

■ Answer

These are the Pages Router data fetching functions — they run on the server and pass data to the page as props. In the App Router, these are replaced by async Server Components.

getStaticProps:

- Runs at BUILD TIME
- Returns props to the page component
- revalidate option enables ISR
- notFound: true → 404 page
- redirect: { destination } → redirect

getServerSideProps:

- Runs on EVERY request
- Has access to req, res, cookies, headers
- Cannot be combined with getStaticProps
- Slower than getStaticProps — no CDN caching

getStaticPaths (for dynamic routes):

- Used with getStaticProps in dynamic routes
- Returns paths array + fallback option
- fallback: false → 404 for unknown paths
- fallback: true → generate on-demand
- fallback: 'blocking' → SSR until generated

App Router equivalent:

- getStaticProps → async Server Component (static by default)
- getServerSideProps → async Server Component + cookies()/headers()

```
// ■■ PAGES ROUTER ■■

// pages/blog/[slug].tsx
import type { GetStaticProps, GetStaticPaths } from 'next'

export const getStaticPaths: GetStaticPaths = async () => {
  const posts = await getPosts()
  return {
    paths: posts.map(p => ({ params: { slug: p.slug } })),
    fallback: 'blocking'  // unknown slugs: SSR once then cache
  }
}

export const getStaticProps: GetStaticProps = async ({ params }) => {
  const post = await getPost(params.slug as string)
  if (!post) return { notFound: true }

  return {
```

```
    props: { post },
```

```
    revalidate: 3600  // ISR: re-generate every hour
  }
}

export default function BlogPost({ post }) {
  return <article>{post.content}</article>
}

// pages/dashboard.tsx — getServerSideProps
import type { GetServerSideProps } from 'next'

export const getServerSideProps: GetServerSideProps = async ({ req, res }) => {
  const token = req.cookies.token
  if (!token) return { redirect: { destination: '/login', permanent: false } }

  const user = await validateToken(token)
  if (!user) return { redirect: { destination: '/login', permanent: false } }

  return { props: { user } }
}

export default function Dashboard({ user }) {
  return <div>Welcome, {user.name}</div>
}

// ■■ APP ROUTER equivalent (cleaner!) ■■
// app/dashboard/page.tsx
import { cookies, redirect } from 'next/headers'

export default async function Dashboard() {
  const token = cookies().get('token')?.value
  if (!token) redirect('/login')
  const user = await validateToken(token)
  if (!user) redirect('/login')
  return <div>Welcome, {user.name}</div>
}
```

■ Interview Tip:
If you're starting a new project, skip getStaticProps/getServerSideProps entirely and use the App Router with async Server Components. They're simpler, more powerful, and composable. getStaticProps and getServerSideProps are legacy API that still work but won't receive new features.

Q18 getStaticPaths vs generateStaticParams — comparison?

■ Answer

Both functions tell Next.js which dynamic routes to pre-render at build time.
getStaticPaths is Pages Router, generateStaticParams is App Router.

getStaticPaths (Pages Router):

- Returns { paths: [...], fallback: ... }
- paths: array of { params: { slug: 'value' } }
- fallback: false | true | 'blocking'
- Must be used with getStaticProps

generateStaticParams (App Router):

- Returns array of param objects directly
- No fallback option — use dynamicParams export instead
- Can be async — directly await data
- Works with layout.tsx too (generates for all children)

fallback vs dynamicParams:

- getStaticPaths fallback: false → dynamicParams = false
- getStaticPaths fallback: true → dynamicParams = true
- getStaticPaths fallback: 'blocking' → dynamicParams = true (App Router default)

Performance note:

- generateStaticParams runs ONCE at build
- Data fetched inside is cached across the page renders
- Much cleaner API — no separate getStaticProps needed

```
// ■■ Pages Router ■■
// pages/products/[id].tsx
export async function getStaticPaths() {
  const products = await getProducts()
  return {
    paths: products.map(p => ({ params: { id: String(p.id) } })),
    fallback: 'blocking'  // ISR: generate unknown ids on demand
  }
}
export async function getStaticProps({ params }) {
  const product = await getProduct(params.id)
  if (!product) return { notFound: true }
  return { props: { product }, revalidate: 60 }
}
export default function Product({ product }) {
  return <div>{product.name}</div>
}

// ■■ App Router equivalent ■■
// app/products/[id]/page.tsx
```

```
export async function generateStaticParams() {
```

```
  const products = await getProducts()
  return products.map(p => ({ id: String(p.id) }))  // simpler!
}

export const dynamicParams = true  // generate unknown ids on demand

export default async function Product({ params }: { params: { id: string } }) {
  const product = await fetch('/api/products/${params.id}', {
    next: { revalidate: 60 }  // ISR equivalent
  }).then(r => r.json())

  if (!product) notFound()
  return <div>{product.name}</div>
}

// Multiple dynamic segments
// app/shop/[category]/[product]/page.tsx
export async function generateStaticParams() {
  const items = await getAllItems()
  return items.map(item => ({
    category: item.category,
    product: item.slug
  }))
}
```

■ Interview Tip:
One major advantage of generateStaticParams: if you call the same fetch inside generateStaticParams and inside the page component, Next.js automatically deduplicates the request. Both receive the same cached response — no double-fetching.

Q19 ISR — Incremental Static Regeneration?

■ Answer

ISR lets you update static pages after build time without rebuilding the entire site.
Pages are initially static (fast CDN delivery) but regenerated in the background.

How ISR works:

- Page is statically generated at build time
- When a request comes in after revalidate seconds, regenerate in background
- Stale-while-revalidate: serve old page while new one generates
- Next visitor gets the fresh page

App Router ISR:

- fetch() with { next: { revalidate: N } }
- Route segment: export const revalidate = N
- N=0 → always dynamic (SSR)
- N=false (or Infinity) → never revalidate (pure SSG)

On-demand revalidation:

- revalidatePath('/blog') — revalidate a specific path
- revalidateTag('posts') — revalidate by cache tag
- Call from Server Action or API route
- Webhook from CMS → revalidate immediately on content change

Time-based vs on-demand:

- Time-based — predictable, simple (news sites, prices)
- On-demand — immediate freshness (CMS, e-commerce)

```
// ■■ TIME-BASED ISR ■■

// Method 1: fetch with revalidate
export default async function ProductPage({ params }) {
  const product = await fetch('https://api.example.com/products/${params.id}', {
    next: { revalidate: 3600 }  // re-generate every 1 hour
  }).then(r => r.json())
  return <div>{product.name}: ${product.price}</div>
}

// Method 2: route segment config
export const revalidate = 3600  // all fetches in this route revalidate every 1h

export default async function Page() {
  const data = await getData()  // uses route-level revalidation
  return <div>{data.title}</div>
}

// ■■ ON-DEMAND REVALIDATION ■■

```

```
// app/api/revalidate/route.ts — webhook endpoint
```

```
import { revalidatePath, revalidateTag } from 'next/cache'
import { NextRequest } from 'next/server'

export async function POST(req: NextRequest) {
  const { secret, path, tag } = await req.json()

  if (secret !== process.env.REVALIDATE_SECRET) {
    return Response.json({ error: 'Invalid secret' }, { status: 401 })
  }

  if (path) revalidatePath(path)         // revalidate specific path
  if (tag) revalidateTag(tag)            // revalidate by cache tag

  return Response.json({ revalidated: true })
}

// Tag-based caching (recommended for precision)
const products = await fetch('/api/products', {
  next: {
    revalidate: 3600,
    tags: ['products', 'product-list']  // cache tags
  }
})

// Later — on-demand revalidation of just products
revalidateTag('products')  // only product pages regenerate
```

■ Interview Tip:
On-demand ISR with webhooks is the production pattern for CMS-driven sites. Contentful, Sanity, and Strapi all support webhooks — when an editor publishes content, they ping your /api/revalidate endpoint, which calls revalidatePath and the page updates instantly.

Q20 Fetching data in Server Components — patterns?

■ Answer

Server Components can fetch data directly — no useEffect, no loading state, no API routes needed.

Basic async Server Component:

- Add async to component function
- await data directly in component body
- Data fetching and rendering collocated

Fetch deduplication:

- Same fetch URL in multiple components = ONE request
- Next.js automatically deduplicates during a render
- Works across generateMetadata + page component

Parallel data fetching:

- await in sequence = waterfall (slow!)
- Promise.all → all fetch in parallel

Data fetching patterns:

- Sequential — when one fetch depends on another
- Parallel — when fetches are independent
- Preload pattern — kick off fetch early
- React cache() — manual memoization across components

Where to fetch:

- As close to where data is used as possible
- Don't prop-drill — each component can fetch its own data

```
// ■■ BASIC async Server Component ■■
export default async function ProductPage({ params }) {
  const product = await db.products.findById(params.id)  // direct DB!
  return <div>{product.name}</div>
}

// ■■ PARALLEL fetching (fast!) ■■
export default async function Dashboard() {
  // ■ Sequential (slow) — each awaits the previous
  // const user = await getUser()
  // const posts = await getPosts()    // waits for user
  // const stats = await getStats()    // waits for posts

  // ■ Parallel — all start simultaneously
  const [user, posts, stats] = await Promise.all([
    getUser(),
    getPosts(),
    getStats()
  ])
  return <div>...</div>
}
```

```
// ■■ React cache() — share data between components ■■
import { cache } from 'react'

// getUser is memoized per request — called multiple times = ONE DB query
const getUser = cache(async (id: string) => {
  return db.users.findById(id)
})

// Layout and Page both call getUser() — only one DB query!
// app/dashboard/layout.tsx
export default async function Layout({ children }) {
  const user = await getUser(getCurrentUserId())  // 1st call → fetches
  return <><UserBadge user={user} />{children}</>
}

// app/dashboard/page.tsx
export default async function Page() {
  const user = await getUser(getCurrentUserId())  // 2nd call → cached!
  return <div>Welcome {user.name}</div>
}

// ■■ Preload pattern ■■
function preloadProduct(id: string) {
  void getProduct(id)  // kick off fetch early, don't await
}

export default async function Page({ params }) {
  preloadProduct(params.id)  // start fetching immediately
  // ... other synchronous work ...
  const product = await getProduct(params.id)  // likely cached by now
  return <Product product={product} />
}
```

■ Interview Tip:
React cache() is the key to sharing server-side data between components without prop drilling. Use it to wrap your data access functions — then both the layout (for the user badge) and the page (for personalization) can call getUser() and only one database query runs.

Q21 Streaming and Suspense in Next.js?

■ Answer

Streaming sends HTML to the browser in chunks as it becomes ready, instead of waiting for everything. Suspense wraps components that aren't ready yet, showing a fallback while they load.

Without streaming:

- Wait for ALL data → generate ALL HTML → send to browser
- User sees blank page until everything is ready

With streaming:

- Send HTML shell immediately
- Stream in each suspended section as data resolves
- User sees content progressively

Suspense in App Router:

- Wrap async components in <Suspense fallback={...}>
- loading.tsx is automatic Suspense wrapper
- Fine-grained control: Suspense at component level

When to add Suspense:

- Slow data fetches that block rendering
- Below-the-fold content
- Independent sections that load at different speeds

use() hook:

- Unwrap a Promise inside a Client Component
- Works with Suspense — component suspends while waiting

```
import { Suspense } from 'react'

// ■■ Page with granular Suspense ■■
export default function DashboardPage() {
  return (
    <div>
      {/* Shows immediately — no async */}
      <DashboardHeader />

      {/* Slow data — shows skeleton while loading */}
      <Suspense fallback={<StatsSkeleton />}>
        <Stats />  {/* async Server Component */}
      </Suspense>

      {/* Independent — loads in parallel */}
      <Suspense fallback={<RecentOrdersSkeleton />}>
        <RecentOrders />
      </Suspense>
    </div>
  )
}
```

```
// Stats.tsx — async component that suspends
async function Stats() {
  const stats = await db.getStats()  // slow query — 500ms
  return <StatsGrid data={stats} />
}

async function RecentOrders() {
  const orders = await db.getOrders()  // different query — 200ms
  return <OrderList orders={orders} />
}
// Stats and RecentOrders stream in AS SOON as their data is ready
// DashboardHeader shows immediately

// ■■ use() hook — for Client Components ■■
'use client'
import { use } from 'react'

function Comments({ commentsPromise }) {
  const comments = use(commentsPromise)  // suspends until resolved
  return <ul>{comments.map(c => <li key={c.id}>{c.text}</li>)}</ul>
}

// Pass promise from Server Component
function Page() {
  const commentsPromise = getComments()  // don't await!
  return (
    <Suspense fallback={<div>Loading comments...</div>}>
      <Comments commentsPromise={commentsPromise} />
    </Suspense>
  )
}
```

■ Interview Tip:
The granular Suspense pattern is the key to great perceived performance. Instead of one big loading state, wrap each independent section in its own Suspense. The header shows instantly, the fast widget loads in 200ms, the slow analytics in 800ms — users see progress, not a blank screen.

Q22 Dynamic Rendering vs Static Rendering?

■ Answer

Next.js automatically chooses static or dynamic rendering based on what your route uses. Understanding this prevents accidental performance regressions.

Static rendering (default):

- Route renders at BUILD time
- Cached in CDN — fastest delivery
- Re-renders with ISR if revalidate is set
- Requires: no dynamic functions, no uncached data fetching

Dynamic rendering (opts in automatically):

- Route renders on every REQUEST
- Cannot be cached at CDN level
- Triggered by using any dynamic function:

- cookies() from next/headers
- headers() from next/headers
- searchParams prop in page
- unstable_noStore() explicitly

Partial Pre-Rendering (Next.js 14 experimental):

- Static shell with dynamic holes
- Best of both worlds
- Shell delivered instantly from CDN
- Dynamic parts stream in per-request

Force rendering mode:

- export const dynamic = 'force-dynamic' — always SSR
- export const dynamic = 'force-static' — always static
- export const dynamic = 'error' — error if dynamic used

```
// ■■ STATIC (default) ■■
export default async function Page() {
  const posts = await fetch('https://api.example.com/posts', {
    next: { revalidate: 3600 }
  }).then(r => r.json())
  return <PostList posts={posts} />
}
// ↑ Static — no dynamic functions used

// ■■ DYNAMIC (automatic opt-in) ■■
import { cookies, headers } from 'next/headers'

export default async function Dashboard() {
  const token = cookies().get('token')?.value  // ← opt-in to dynamic!
  const ua = headers().get('user-agent')       // ← also opts-in
  const user = await getUser(token)
```

```
  return <div>Welcome, {user.name}</div>
```

```
18 }
19 // ↑ Dynamic — renders on every request
20
21 // searchParams also opts in to dynamic
22 export default function SearchPage({
23   searchParams  // ← using this opts into dynamic rendering
24 }: {
25   searchParams: { q: string }
26 }) {
27   return <SearchResults query={searchParams.q} />
28 }
29
30 // ■■ Force modes ■■
31 export const dynamic = 'force-dynamic'  // always SSR
32 export const dynamic = 'force-static'   // error if dynamic used
33
34 // ■■ Partial Pre-Rendering (PPR) — experimental ■■
35 // Wrap dynamic content in Suspense
36 export default function Page() {
37   return (
38     <div>
39       <StaticHeader />  {/* included in static shell */}
40       <Suspense fallback={<Skeleton />}>
41         <DynamicUserSection />  {/* streams in dynamically */}
42       </Suspense>
43     </div>
44   )
45 }
```

■ Interview Tip:
A common mistake: accidentally making a page dynamic by reading cookies in a helper function that's imported but not actually called with user data. Always check your build output — static pages show ■ (circle), dynamic pages show λ (lambda). Aim for as many ■ as possible.

Q23 Revalidation and Cache Control in Next.js?

■ Answer

Next.js has a multi-layer caching system that controls how and when data is refreshed. Understanding the cache layers prevents stale data bugs and over-fetching.

Cache layers (newest to oldest):

- Request Memoization — same fetch() deduped per render
- Data Cache — fetch() results persisted across requests
- Full Route Cache — rendered HTML cached on server
- Router Cache — client-side cache of visited routes

Revalidation options for fetch():

- next: { revalidate: N } — time-based (seconds)
- next: { tags: ['tag'] } — tag-based
- cache: 'no-store' — never cache, always fetch fresh
- cache: 'force-cache' — always use cache (default)

Programmatic revalidation:

- revalidatePath(path, 'page' | 'layout') — by path
- revalidateTag(tag) — all fetches with that tag
- Both work from Server Actions and Route Handlers

Opting out of all caching:

- fetch with cache: 'no-store'
- noStore() from next/cache
- Using cookies(), headers(), or searchParams

```
import { unstable_noStore as noStore, revalidatePath, revalidateTag } from 'next/cache'

// ■■ CACHE OPTIONS ■■

// Default — cached until revalidated
const data = await fetch('/api/data')  // force-cache by default

// Time-based ISR
const products = await fetch('/api/products', {
  next: { revalidate: 3600 }  // stale after 1 hour
})

// Tagged for on-demand revalidation
const posts = await fetch('/api/posts', {
  next: {
    revalidate: 86400,  // also time-based
    tags: ['posts', 'blog-content']
  }
})

```

```
// Never cache — always fresh
```

```
22 const livePrice = await fetch('/api/price', {
23   cache: 'no-store'  // real-time data
24 })
25
26 // noStore() — opts current component out of caching
27 export default async function LiveFeed() {
28   noStore()  // this component always re-renders
29   const feed = await getFeed()
30   return <Feed items={feed} />
31 }
32
33 // ■■ ON-DEMAND REVALIDATION ■■
34
35 // From a Server Action (CMS webhook handler)
36 'use server'
37 export async function publishPost(postId: string) {
38   await db.posts.update({ where: { id: postId }, data: { published: true } })
39
40   // Revalidate all pages that show posts
41   revalidatePath('/blog')               // blog listing
42   revalidatePath('/blog/${postId}')     // specific post
43   revalidateTag('posts')                // all tagged fetches
44 }
45
46 // From API Route (webhook from Contentful/Sanity)
47 export async function POST(req: Request) {
48   const { type, data } = await req.json()
49   if (type === 'content.publish') {
50     revalidateTag('cms-content')
51   }
52   return Response.json({ revalidated: true })
53 }
```

■ Interview Tip:
The Router Cache (client-side) caches visited pages for 30 seconds (static) or 0 seconds (dynamic). This means navigating back to a page shows the cached version — great for performance, but confusing when data changes. Call router.refresh() to manually clear the Router Cache.

Q24 Edge Runtime vs Node.js Runtime in Next.js?

■ Answer

Next.js supports two server runtimes: Node.js (default) and Edge (lightweight, globally distributed).

Node.js Runtime (default):

- Full Node.js API available
- Can use: fs, crypto, child_process, native modules
- npm packages that use Node.js APIs
- Runs in a single region (unless distributed manually)
- Larger cold starts (~100-500ms on Lambda)
- Use for: most API routes, complex logic, DB connections

Edge Runtime:

- Restricted Web API only (no Node.js built-ins)
- Runs on Cloudflare Workers / Vercel Edge Network
- Globally distributed — closest to user
- Near-zero cold starts (~5ms)
- Memory limit: 128MB
- Use for: middleware, simple transformations, A/B testing, auth checks

How to set runtime:

- export const runtime = 'edge' — in any route or layout
- Default is 'nodejs'

What Edge Runtime CAN use:

- fetch, Request, Response, URL
- Web Crypto API
- TextEncoder, TextDecoder
- Readable/WritableStream

```
// ■■ EDGE RUNTIME ■■

// middleware.ts — Edge by default (ALWAYS runs at the edge)
import { NextResponse } from 'next/server'
import type { NextRequest } from 'next/server'

export function middleware(req: NextRequest) {
  // Check auth, redirect, add headers — all at the edge (fast!)
  const token = req.cookies.get('token')?.value
  if (!token && req.nextUrl.pathname.startsWith('/dashboard')) {
    return NextResponse.redirect(new URL('/login', req.url))
  }
  return NextResponse.next()
}

// API route running at Edge
// app/api/geo/route.ts
```

```
export const runtime = 'edge'
```

```
export async function GET(req: Request) {
  // Vercel provides geo info in headers at edge
  const country = req.headers.get('x-vercel-ip-country') || 'US'
  return Response.json({ country })
}

// ■■ NODE.JS RUNTIME (default) ■■

// app/api/users/route.ts — Node.js runtime (default)
// Can use Prisma, bcrypt, sharp, etc.
import { PrismaClient } from '@prisma/client'

const prisma = new PrismaClient()

export async function GET() {
  const users = await prisma.user.findMany()  // Node.js native package
  return Response.json(users)
}

// Force Node.js runtime explicitly
export const runtime = 'nodejs'  // explicit

// ■ This would ERROR at edge runtime:
// import fs from 'fs'  // Node.js built-in — not available at edge
// import bcrypt from 'bcrypt'  // native addon — not available at edge

// ■ Edge-compatible auth
// Use jose (pure JS JWT) instead of jsonwebtoken (Node.js)
```

■ Interview Tip:
Put your auth checks in middleware (Edge Runtime) — it runs before any server-side rendering happens, adding ~0ms latency since it's geographically close to the user. Reserve Node.js runtime for database queries, file processing, and npm packages that need Node.js APIs.

Q25 Route Handlers — app/api with route.ts?

■ Answer

Route Handlers are API endpoints in the App Router — they replace pages/api/ from the Pages Router.

File convention:

- app/api/users/route.ts → /api/users
- app/api/users/[id]/route.ts → /api/users/:id
- File MUST be named route.ts (not route.tsx)

Export HTTP methods:

- export function GET(req) {}
- export function POST(req) {}
- export function PUT(req) {}
- export function DELETE(req) {}
- export function PATCH(req) {}

Request object:

- req.json() — parse JSON body
- req.formData() — parse form data
- req.text() — raw text
- req.headers — Headers object
- req.nextUrl — parsed URL with searchParams
- req.cookies — cookies (NextRequest only)

Response:

- Return new Response(body, { status, headers })
- Return NextResponse.json(data, { status })

Caching:

- GET handlers are cached by default
- Add dynamic = 'force-dynamic' to disable

```
// app/api/users/route.ts
import { NextRequest, NextResponse } from 'next/server'
import { db } from '@/lib/db'

// GET /api/users?page=1&limit=20
export async function GET(req: NextRequest) {
  const { searchParams } = req.nextUrl
  const page = parseInt(searchParams.get('page') || '1')
  const limit = parseInt(searchParams.get('limit') || '20')

  const users = await db.user.findMany({
    skip: (page - 1) * limit,
    take: limit
  })
  return NextResponse.json({ users, page, limit })
```

```
}
```

```
// POST /api/users
export async function POST(req: NextRequest) {
  const body = await req.json()
  const { name, email, password } = body

  // Validate
  if (!name || !email) {
    return NextResponse.json({ error: 'Name and email required' }, { status: 400 })
  }

  const user = await db.user.create({ data: { name, email } })
  return NextResponse.json(user, { status: 201 })
}

// app/api/users/[id]/route.ts
export async function GET(
  req: NextRequest,
  { params }: { params: { id: string } }
) {
  const user = await db.user.findUnique({ where: { id: params.id } })
  if (!user) return NextResponse.json({ error: 'Not found' }, { status: 404 })
  return NextResponse.json(user)
}

export async function DELETE(
  req: NextRequest,
  { params }: { params: { id: string } }
) {
  await db.user.delete({ where: { id: params.id } })
  return new Response(null, { status: 204 })
}
```

■ Interview Tip:
Route Handlers are a great escape hatch for webhooks, OAuth callbacks, file uploads, and anything that needs raw HTTP control. For regular data mutations triggered by user actions, prefer Server Actions — they're simpler (no API URL, automatic CSRF protection, works without JavaScript).

Q26 GET, POST, PUT, DELETE handlers — complete patterns?

■ Answer

A complete CRUD API route implementation with validation, error handling, and authentication.

RESTful convention with Route Handlers:

- GET /api/posts — list all
- POST /api/posts — create new
- GET /api/posts/[id] — get one
- PUT /api/posts/[id] — replace one
- PATCH /api/posts/[id] — partial update
- DELETE /api/posts/[id] — delete one

Common patterns:

- Auth middleware → extract user from token
- Validate with Zod before DB operation
- Return consistent error format
- Use correct HTTP status codes
- Log errors server-side

```
// lib/api-helpers.ts — reusable utilities
import { NextRequest, NextResponse } from 'next/server'
import { z } from 'zod'

export function apiError(message: string, status = 400) {
  return NextResponse.json({ error: message }, { status })
}

export async function getAuthUser(req: NextRequest) {
  const token = req.headers.get('authorization')?.split(' ')[1]
  if (!token) return null
  try { return verifyToken(token) }
  catch { return null }
}

// app/api/posts/route.ts
const CreatePostSchema = z.object({
  title: z.string().min(1).max(200),
  content: z.string().min(10),
  published: z.boolean().default(false)
})

export async function GET(req: NextRequest) {
  const posts = await db.post.findMany({
    where: { published: true },
    orderBy: { createdAt: 'desc' },
    take: 20
  })
  return NextResponse.json(posts)
```

```
}
```

```
31
32 export async function POST(req: NextRequest) {
33   const user = await getAuthUser(req)
34   if (!user) return apiError('Unauthorized', 401)
35
36   const body = await req.json()
37   const result = CreatePostSchema.safeParse(body)
38   if (!result.success) return apiError(result.error.message)
39
40   const post = await db.post.create({
41     data: { ...result.data, authorId: user.id }
42   })
43   return NextResponse.json(post, { status: 201 })
44 }
45
46 // app/api/posts/[id]/route.ts
47 export async function PUT(req: NextRequest, { params }) {
48   const user = await getAuthUser(req)
49   if (!user) return apiError('Unauthorized', 401)
50
51   const post = await db.post.findUnique({ where: { id: params.id } })
52   if (!post) return apiError('Not found', 404)
53   if (post.authorId !== user.id) return apiError('Forbidden', 403)
54
55   const body = await req.json()
56   const result = CreatePostSchema.safeParse(body)
57   if (!result.success) return apiError(result.error.message)
58
59   const updated = await db.post.update({
60     where: { id: params.id },
61     data: result.data
62   })
63   return NextResponse.json(updated)
64 }
65
66 export async function DELETE(req: NextRequest, { params }) {
67   const user = await getAuthUser(req)
68   if (!user) return apiError('Unauthorized', 401)
69   await db.post.delete({ where: { id: params.id, authorId: user.id } })
70   return new Response(null, { status: 204 })
71 }
```

■ Interview Tip:
Always validate request bodies with Zod before touching the database. A single missing .safeParse() check is a path to crashes or SQL injection. Create a reusable validate() helper that returns the error response automatically — makes route handlers much cleaner.

Q27 Dynamic API Routes and request/response objects?

■ Answer

Dynamic API routes and understanding the Next.js request/response API.

Dynamic route handlers:

- app/api/users/[id]/route.ts — receives params as second arg
- app/api/[...path]/route.ts — catch-all

NextRequest (extends Request):

- req.nextUrl — URL object with searchParams
- req.cookies — cookies (Cookies object)
- req.geo — country, city, region (Vercel)
- req.ip — client IP
- All standard Request methods: json(), text(), formData()

NextResponse (extends Response):

- NextResponse.json(data, options) — JSON response
- NextResponse.redirect(url) — redirect
- NextResponse.rewrite(url) — rewrite
- NextResponse.next() — continue (middleware only)
- Set cookies with response.cookies.set()

Headers in responses:

- Set via Response constructor or NextResponse
- CORS, cache control, content-type

```
// app/api/users/[id]/orders/[orderId]/route.ts
import { NextRequest, NextResponse } from 'next/server'

export async function GET(
  req: NextRequest,
  { params }: { params: { id: string; orderId: string } }
) {
  const { id: userId, orderId } = params

  // Query params
  const { searchParams } = req.nextUrl
  const include = searchParams.get('include')?.split(',') || []

  // Headers
  const authHeader = req.headers.get('authorization')
  const contentType = req.headers.get('content-type')

  // Cookies
  const sessionCookie = req.cookies.get('session')?.value

  const order = await db.order.findFirst({
    where: { id: orderId, userId }
```

```
  })
```

```
  if (!order) return NextResponse.json({ error: 'Not found' }, { status: 404 })

  return NextResponse.json(order)
}

// Setting cookies in response
export async function POST(req: NextRequest) {
  const { email, password } = await req.json()
  const user = await authenticate(email, password)

  const response = NextResponse.json({ user })

  // Set cookie on response
  response.cookies.set('token', generateToken(user), {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'strict',
    maxAge: 60 * 60 * 24 * 7  // 1 week
  })

  return response
}

// Custom headers
export async function GET() {
  const data = await getData()
  return new Response(JSON.stringify(data), {
    status: 200,
    headers: {
      'Content-Type': 'application/json',
      'Cache-Control': 'public, s-maxage=60, stale-while-revalidate=300',
      'Access-Control-Allow-Origin': '*'
    }
  })
}
```

■ Interview Tip:
Set cookies on NextResponse when you need to set them from Route Handlers (like login). For Server Actions, use cookies() from next/headers instead. Both ways set HTTP-only cookies that JavaScript can't access — always use httpOnly: true for auth cookies.

Q28 Middleware in Next.js — how it works?

■ Answer

Next.js middleware runs before every request is processed — at the edge, before caching.
It's the ideal place for: auth checks, redirects, A/B testing, bot detection.

middleware.ts:

- Place at project root (next to app/)
- Runs on EVERY request matching the config matcher
- Runs at Edge Runtime by default
- Has access to: request, cookies, headers

Middleware can:

- Redirect to another URL
- Rewrite the URL (internal proxy)
- Add/modify headers
- Read/set cookies
- Return a response directly

config.matcher:

- Specify which routes middleware applies to
- Regex-like path patterns
- Exclude static files and API routes you don't need

Important limitations:

- Cannot access databases (Edge Runtime)
- Cannot use Node.js built-ins
- Keep it fast — runs on every request

```
// middleware.ts — root of project
import { NextResponse } from 'next/server'
import type { NextRequest } from 'next/server'
import { verifyToken } from '@/lib/auth'

export async function middleware(req: NextRequest) {
  const { pathname } = req.nextUrl

  // ■■ Auth check for protected routes
  const isProtected = pathname.startsWith('/dashboard') ||
    pathname.startsWith('/settings') ||
    pathname.startsWith('/api/protected')

  if (isProtected) {
    const token = req.cookies.get('token')?.value

    if (!token) {
      const loginUrl = new URL('/login', req.url)
      loginUrl.searchParams.set('from', pathname)  // redirect back after login
```

```
      return NextResponse.redirect(loginUrl)
```

```
    }

    try {
      const user = await verifyToken(token)  // must be Edge-compatible!
      // Add user info to headers for downstream components
      const response = NextResponse.next()
      response.headers.set('x-user-id', user.id)
      response.headers.set('x-user-role', user.role)
      return response
    } catch {
      return NextResponse.redirect(new URL('/login', req.url))
    }
  }

  // ■■ A/B testing
  const bucket = req.cookies.get('ab-bucket')?.value ||
    (Math.random() < 0.5 ? 'a' : 'b')

  if (pathname === '/' && bucket === 'b') {
    const response = NextResponse.rewrite(new URL('/landing-b', req.url))
    response.cookies.set('ab-bucket', 'b', { maxAge: 60 * 60 * 24 * 7 })
    return response
  }

  return NextResponse.next()
}

// Apply to specific paths only
export const config = {
  matcher: [
    '/dashboard/:path*',
    '/settings/:path*',
    '/api/protected/:path*',
    '/((?!_next/static|_next/image|favicon.ico|public).*)',
  ]
}
```

■ Interview Tip:
Pass user info to Server Components via middleware headers (x-user-id, x-user-role) rather than decoding the token again in every page. Then read these headers in your server components with headers().get('x-user-id') — fast and avoids repeated JWT verification.

Q29 Authentication with NextAuth.js?

■ Answer

NextAuth.js (Auth.js) is the standard authentication library for Next.js.
It handles OAuth providers, email/password, JWT sessions, and database sessions.

Setup (Auth.js v5 — App Router):

- Create auth.ts config file
- Add route handler in app/api/auth/[...nextauth]/route.ts
- Wrap app with SessionProvider (if using useSession)

Providers:

- OAuth: Google, GitHub, Discord, Twitter, etc.
- Credentials: email + password (custom validation)
- Email: magic link

Session strategies:

- JWT (default) — session in cookie, no DB needed
- Database — session stored in DB (needs adapter)

Accessing session:

- Server: auth() from auth.ts
- Client: useSession() from next-auth/react
- Middleware: auth() or token from cookies

Callbacks:

- jwt() — customize token (add user role, id)
- session() — expose token data to client
- signIn() — allow/deny sign in
- redirect() — customize redirect after sign in

```
// auth.ts — central config
import NextAuth from 'next-auth'
import Google from 'next-auth/providers/google'
import Credentials from 'next-auth/providers/credentials'
import { PrismaAdapter } from '@auth/prisma-adapter'
import { prisma } from '@/lib/prisma'
import bcrypt from 'bcryptjs'

export const { handlers, signIn, signOut, auth } = NextAuth({
  adapter: PrismaAdapter(prisma),
  providers: [
    Google({
      clientId: process.env.GOOGLE_CLIENT_ID!,
      clientSecret: process.env.GOOGLE_CLIENT_SECRET!
    }),
    Credentials({
      credentials: {
```

```
      email: { type: 'email' },
```

```
19          password: { type: 'password' }
20        },
21        async authorize({ email, password }) {
22          const user = await prisma.user.findUnique({ where: { email } })
23          if (!user || !user.password) return null
24          const valid = await bcrypt.compare(password, user.password)
25          return valid ? user : null
26        }
27      })
28    ],
29    callbacks: {
30      jwt({ token, user }) {
31        if (user) { token.role = user.role; token.id = user.id }
32        return token
33      },
34      session({ session, token }) {
35        session.user.id = token.id as string
36        session.user.role = token.role as string
37        return session
38      }
39    },
40    pages: { signIn: '/login', error: '/auth/error' }
41  })
42
43  // app/api/auth/[...nextauth]/route.ts
44  import { handlers } from '@/auth'
45  export const { GET, POST } = handlers
46
47  // Server Component — get session
48  import { auth } from '@/auth'
49  export default async function Page() {
50    const session = await auth()  // server-side!
51    if (!session) redirect('/login')
52    return <div>Hello {session.user.name}</div>
53  }
54
55  // Client Component
56  'use client'
57  import { useSession, signIn, signOut } from 'next-auth/react'
58  function AuthButton() {
59    const { data: session } = useSession()
60    return session
61      ? <button onClick={() => signOut()}>Sign Out</button>
62      : <button onClick={() => signIn('google')}>Sign In with Google</button>
63  }
```

■ Interview Tip:
Use the server-side auth() function from auth.ts in Server Components — it's more efficient than useSession because it doesn't add client-side JavaScript. Reserve useSession() for Client Components that need to react to session changes in real-time.

Q30 Cookies and Headers in Next.js?

■ Answer

Next.js provides server-side APIs for reading and setting cookies and HTTP headers.

Reading cookies and headers (server):

- cookies() — from next/headers, Server Components + Server Actions
- headers() — from next/headers, same
- Both are request-scoped (per-request values)
- Reading them opts the route into DYNAMIC rendering

Setting cookies:

- In Server Actions: cookies().set(name, value, options)
- In Route Handlers: response.cookies.set()
- In Middleware: response.cookies.set()

Deleting cookies:

- cookies().delete(name) in Server Actions
- cookies().set(name, '', { maxAge: 0 })

Security best practices:

- httpOnly: true — prevent XSS
- secure: true — HTTPS only
- sameSite: 'lax' or 'strict' — CSRF protection
- Set maxAge or expires for session management

```
import { cookies, headers } from 'next/headers'

// ■■ Reading in Server Components ■■
export default async function Dashboard() {
  // Cookies
  const cookieStore = cookies()
  const theme = cookieStore.get('theme')?.value || 'light'
  const authToken = cookieStore.get('auth-token')?.value
  const allCookies = cookieStore.getAll()

  // Headers
  const headersList = headers()
  const userAgent = headersList.get('user-agent')
  const acceptLang = headersList.get('accept-language')
  const xForwardedFor = headersList.get('x-forwarded-for')  // real IP

  return <div data-theme={theme}>...</div>
}

// ■■ Setting cookies in Server Actions ■■
'use server'
import { cookies } from 'next/headers'

```

```
export async function login(formData: FormData) {
```

```
25    const email = formData.get('email') as string
26    const password = formData.get('password') as string
27
28    const user = await authenticate(email, password)
29    if (!user) throw new Error('Invalid credentials')
30
31    // Set auth cookie
32    cookies().set('token', generateJWT(user), {
33      httpOnly: true,
34      secure: process.env.NODE_ENV === 'production',
35      sameSite: 'lax',
36      maxAge: 60 * 60 * 24 * 7,  // 7 days
37      path: '/'
38    })
39
40    // Set theme preference
41    cookies().set('theme', 'dark', { maxAge: 60 * 60 * 24 * 365 })
42  }
43
44  export async function logout() {
45    cookies().delete('token')  // or set maxAge: 0
46    redirect('/login')
47  }
48
49  // ■■ In Route Handler ■■
50  export async function POST(req: NextRequest) {
51    const response = NextResponse.json({ success: true })
52    response.cookies.set('session', createSession(), {
53      httpOnly: true, secure: true, sameSite: 'lax'
54    })
55    return response
56  }
```

■ Interview Tip:
The cookies() function from next/headers is only available in Server Components, Server Actions, and Route Handlers. Never call it in Client Components — use document.cookie for client-side reading, but only for non-sensitive data. Sensitive auth cookies should always be httpOnly (no JS access).

Q31 CORS in Next.js — how to handle it?

■ Answer

CORS (Cross-Origin Resource Sharing) controls which origins can call your Next.js API.

When you need CORS:

- Your Next.js API is called from a DIFFERENT domain
- Same origin (calling from your own Next.js frontend) = no CORS needed
- Third-party apps, mobile apps, or other domains calling your API

CORS headers needed:

- Access-Control-Allow-Origin — which origins allowed
- Access-Control-Allow-Methods — allowed HTTP methods
- Access-Control-Allow-Headers — allowed request headers
- Access-Control-Allow-Credentials — allow cookies

Preflight requests (OPTIONS):

- Browser sends OPTIONS before actual request
- Must handle OPTIONS method in route handlers

Implementation options:

- next.config.js headers() — global CORS
- Route-level CORS — per API route
- Middleware — centralized CORS handling
- cors npm package — helper utilities

```
// ■■ OPTION 1: next.config.js (global) ■■
module.exports = {
  async headers() {
    return [
      {
        source: '/api/:path*',
        headers: [
          { key: 'Access-Control-Allow-Origin', value: 'https://app.example.com' },
          { key: 'Access-Control-Allow-Methods', value: 'GET,POST,PUT,DELETE,OPTIONS' },
          { key: 'Access-Control-Allow-Headers', value: 'Content-Type, Authorization' }
        ]
      }
    ]
  }
}

// ■■ OPTION 2: Route Handler level ■■
const CORS_HEADERS = {
  'Access-Control-Allow-Origin': process.env.ALLOWED_ORIGIN || '*',
  'Access-Control-Allow-Methods': 'GET, POST, PUT, DELETE, OPTIONS',
  'Access-Control-Allow-Headers': 'Content-Type, Authorization',
}

```

```
// Handle preflight
```

```
25 export function OPTIONS() {
26   return new Response(null, { status: 204, headers: CORS_HEADERS })
27 }
28
29 export async function GET() {
30   const data = await getData()
31   return NextResponse.json(data, { headers: CORS_HEADERS })
32 }
33
34 // ■■ OPTION 3: Middleware (centralized) ■■
35 // middleware.ts
36 export function middleware(req: NextRequest) {
37   if (req.method === 'OPTIONS') {
38     return new Response(null, {
39       status: 204,
40       headers: {
41         'Access-Control-Allow-Origin': getAllowedOrigin(req),
42         'Access-Control-Allow-Methods': 'GET, POST, PUT, DELETE, OPTIONS',
43         'Access-Control-Allow-Headers': 'Content-Type, Authorization',
44         'Access-Control-Max-Age': '86400'
45       }
46     })
47   }
48   const response = NextResponse.next()
49   response.headers.set('Access-Control-Allow-Origin', getAllowedOrigin(req))
50   return response
51 }
52
53 function getAllowedOrigin(req: NextRequest) {
54   const origin = req.headers.get('origin') || ''
55   const allowed = process.env.ALLOWED_ORIGINS?.split(',') || []
56   return allowed.includes(origin) ? origin : allowed[0] || '*'
57 }
```

■ Interview Tip:
Never use Access-Control-Allow-Origin: * with credentials (cookies). If your API uses cookies for auth, you must specify the exact origin and set Access-Control-Allow-Credentials: true. Wildcard + credentials = browsers block the request anyway.

Q32 Performance Optimization in Next.js?

■ Answer

Performance in Next.js involves choosing the right rendering strategy, optimizing assets, and measuring.

Core Web Vitals targets:

- LCP (Largest Contentful Paint) < 2.5s
- FID/INP (Interaction to Next Paint) < 200ms
- CLS (Cumulative Layout Shift) < 0.1

Rendering optimizations:

- Use SSG/ISR for public pages (CDN delivery)
- Minimize 'use client' boundaries
- Push client boundaries to leaves of component tree
- Stream with Suspense instead of waterfall loading

Asset optimizations:

- next/image — lazy loading, WebP, responsive sizes
- next/font — zero layout shift, self-hosted fonts
- next/script — controlled script loading strategy

Bundle optimizations:

- Dynamic imports — code split on demand
- Analyze bundle: @next/bundle-analyzer
- Tree-shake: avoid importing entire libraries

Database/API optimizations:

- React cache() — deduplicate server-side fetches
- Parallel data fetching with Promise.all
- Edge runtime for latency-sensitive routes

```
// ■■ FONT OPTIMIZATION ■■
import { Inter, Roboto_Mono } from 'next/font/google'

const inter = Inter({
  subsets: ['latin'],
  display: 'swap',  // show fallback while loading
  variable: '--font-inter'  // CSS variable
})

// app/layout.tsx
export default function Layout({ children }) {
  return (
    <html lang='en' className={inter.variable}>
      <body>{children}</body>
    </html>
  )
}
```

```
// ■■ SCRIPT OPTIMIZATION ■■
import Script from 'next/script'

export default function Page() {
  return (
    <>
      {/* afterInteractive — after hydration (analytics) */}
      <Script src='https://www.googletagmanager.com/gtag/js' strategy='afterInteractive' />

      {/* lazyOnload — low priority (chat widgets) */}
      <Script src='https://widget.intercom.io/widget.js' strategy='lazyOnload' />

      {/* beforeInteractive — critical, before page hydrates */}
      <Script id='theme-init' strategy='beforeInteractive'>
        {'document.documentElement.className = localStorage.getItem('theme') || 'light''}
      </Script>
    </>
  )
}

// ■■ BUNDLE ANALYZER ■■
// npm install @next/bundle-analyzer
const withBundleAnalyzer = require('@next/bundle-analyzer')({
  enabled: process.env.ANALYZE === 'true'
})
module.exports = withBundleAnalyzer(nextConfig)
// ANALYZE=true npm run build

// ■■ MINIMIZE CLIENT BOUNDARIES ■■
// ■ Bad: entire tree is client-side
'use client'
export default function ProductPage({ product }) {
  const [liked, setLiked] = useState(false)
  return (
    <div><h1>{product.title}</h1>  {/* static */}
      <p>{product.description}</p>  {/* static */}
      <button onClick={() => setLiked(l => !l)}>{liked ? '♥' : '■'}</button>
    </div>
  )
}

// ■ Good: only the interactive part is client
function LikeButton() { 'use client'; /* ... */ }
export default async function ProductPage({ params }) {
  const product = await getProduct(params.id)  // server!
  return <div><h1>{product.title}</h1><LikeButton /></div>
}
```

■ Interview Tip:
The single biggest performance win in Next.js: push 'use client' as deep into the component tree as possible. A page where only the like button is a Client Component is much faster than a page where the entire product layout is client-side.

Q33 Code Splitting and Lazy Loading in Next.js?

■ Answer

Code splitting breaks your JavaScript into smaller chunks loaded on demand.
Next.js does automatic code splitting per page — plus dynamic imports for finer control.

Automatic code splitting:

- Next.js automatically splits code per route
- Each page gets its own JS bundle
- Shared code goes into a common chunk
- You get this for free — no configuration

Dynamic imports (next/dynamic):

- Load component only when needed
- Reduces initial bundle size
- Shows fallback while component loads
- Can disable SSR for browser-only components

When to use dynamic imports:

- Heavy components (chart libraries, editors, maps)
- Components only visible on user action (modals)
- Browser-only components (window/document usage)
- Third-party widgets loaded conditionally

next/dynamic options:

- loading — fallback component while loading
- ssr: false — skip server-side rendering
- suspense: true — use Suspense instead of loading prop

```
import dynamic from 'next/dynamic'

// ■■ Basic dynamic import ■■
const HeavyChart = dynamic(() => import('@/components/HeavyChart'), {
  loading: () => <div className='h-64 animate-pulse bg-gray-200 rounded' />,
  ssr: false  // chart uses window — skip SSR
})

const MarkdownEditor = dynamic(() => import('@/components/MDEditor'), {
  loading: () => <p>Loading editor...</p>,
  ssr: false  // Monaco editor is browser-only
})

// Use like normal components — loads when rendered
export default function Dashboard() {
  const [showChart, setShowChart] = useState(false)
  return (
    <div>
      <button onClick={() => setShowChart(true)}>Show Analytics</button>
```

```
      {showChart && <HeavyChart />}  {/* loads ONLY when showChart is true */}
```

```
      </div>
    )
  }

  // ■■ Dynamic import with named export ■■
  const Modal = dynamic(() =>
    import('@/components/Modal').then(mod => mod.Modal)  // named export
  )

  // ■■ Using Suspense (App Router preferred) ■■
  const MapComponent = dynamic(() => import('@/components/Map'), {
    suspense: true  // use Suspense instead
  })

  function Page() {
    return (
      <Suspense fallback={<MapSkeleton />}>
        <MapComponent />
      </Suspense>
    )
  }

  // ■■ Import libraries dynamically ■■
  // Don't import at top level if only used sometimes
  async function exportToPDF() {
    const { jsPDF } = await import('jspdf')  // loads only when called
    const doc = new jsPDF()
    doc.save('report.pdf')
  }

  // ■■ Preload on hover ■■
  const UserProfile = dynamic(() => import('./UserProfile'))

  function UserLink({ userId }) {
    return (
      <a
        href={'/users/${userId}'}
        onMouseEnter={() => UserProfile.preload()}  // preload on hover
      >
        View Profile
      </a>
    )
  }
```

■ Interview Tip:
Use dynamic imports aggressively for chart libraries (Recharts, Chart.js, D3) and rich text editors (TipTap, Quill, Monaco). These can add 200-500KB to your bundle. Lazy loading them means users who never see the chart don't pay the cost of downloading it.

Q34 Prefetching and Preloading in Next.js?

■ Answer

Next.js automatically prefetches pages linked with <Link> — making navigation feel instant.

Automatic prefetching:

- <Link> prefetches linked pages when they enter viewport
- In production only — disabled in development
- For static pages: prefetches the full page
- For dynamic pages: prefetches shared layout JS only

Prefetch control:

- prefetch={false} — disable for rare links
- prefetch={true} — force prefetch (even off-screen)
- Default: prefetches when in viewport

Resource hints (next/head in Pages Router):

- <link rel='preload'> — critical resources
- <link rel='prefetch'> — future navigation resources

Image preloading:

- priority prop on next/image — adds preload link
- Use for LCP images (hero images, above fold)

Script preloading:

- next/script strategy='beforeInteractive' — critical scripts
- strategy='afterInteractive' — analytics, etc.

```
// ■■ Link prefetching ■■
import Link from 'next/link'

function Navigation() {
  return (
    <nav>
      {/* Prefetches /about when link enters viewport */}
      <Link href='/about'>About</Link>

      {/* Disable prefetch for rare page */}
      <Link href='/admin' prefetch={false}>Admin</Link>

      {/* Force prefetch immediately (not just viewport) */}
      <Link href='/dashboard' prefetch={true}>Dashboard</Link>
    </nav>
  )
}

// ■■ Programmatic prefetch ■■
'use client'
import { useRouter } from 'next/navigation'
```

```
function ProductCard({ product }) {
  const router = useRouter()

  return (
    <div
      onMouseEnter={() => router.prefetch('/products/${product.id}')}
      onClick={() => router.push('/products/${product.id}')}
    >
      {product.name}
    </div>
  )
}

// ■■ Image preloading (LCP) ■■
import Image from 'next/image'

export default function HeroSection() {
  return (
    <Image
      src='/hero.jpg'
      alt='Hero'
      width={1920}
      height={1080}
      priority  // ← preloads this image (adds <link rel='preload'>)
    />
  )
}

// ■■ Preload data with React cache ■■
import { cache } from 'react'

const getProduct = cache(async (id: string) => {
  return db.product.findUnique({ where: { id } })
})

// Kick off fetch early in parent, child gets cached result
export default async function ProductPage({ params }) {
  void getProduct(params.id)  // start fetching immediately
  // ... other work ...
  const product = await getProduct(params.id)  // returns cached
  return <ProductDetail product={product} />
}
```

■ Interview Tip:
The router.prefetch() on mouseEnter pattern is used by major sites to make navigation feel instant. When a user hovers over a link (~100ms before clicking), the page JavaScript is already downloading. By the time they click, it's ready.

Q35 Environment Variables in Next.js?

■ Answer

Next.js has a structured approach to environment variables with clear client/server boundaries.

Variable types:

- NEXT_PUBLIC_ prefix — available on client AND server
- No prefix — server-only (NEVER in browser bundle)

.env files (priority order):

- .env.local — local overrides (gitignored)
- .env.development — development only
- .env.production — production only
- .env — all environments (committed to git)

Accessing variables:

- Server: process.env.DATABASE_URL
- Client: process.env.NEXT_PUBLIC_API_URL
- process.env is NOT available in client components for server vars

Runtime vs Build-time:

- NEXT_PUBLIC_ vars are inlined at BUILD time
- Changing them requires a rebuild
- For runtime config: use a /api/config endpoint

Validation:

- Validate at startup with Zod (crash early if missing)
- Create a validated env.ts module

```
// ■■ .env.local ■■
// DATABASE_URL=postgresql://localhost/myapp
// JWT_SECRET=super-secret-key-min-32-chars-long
// NEXT_PUBLIC_API_URL=http://localhost:3000
// NEXT_PUBLIC_STRIPE_PK=pk_test_...

// ■■ Validated env.ts ■■
import { z } from 'zod'

const serverEnvSchema = z.object({
  DATABASE_URL: z.string().url(),
  JWT_SECRET: z.string().min(32),
  NEXTAUTH_SECRET: z.string().min(32),
  STRIPE_SECRET_KEY: z.string().startsWith('sk_'),
})

const clientEnvSchema = z.object({
  NEXT_PUBLIC_API_URL: z.string().url(),
  NEXT_PUBLIC_STRIPE_PK: z.string().startsWith('pk_'),
})
```

```
// Validate at import time — fails fast if something's wrong
export const serverEnv = serverEnvSchema.parse(process.env)
export const clientEnv = clientEnvSchema.parse({
  NEXT_PUBLIC_API_URL: process.env.NEXT_PUBLIC_API_URL,
  NEXT_PUBLIC_STRIPE_PK: process.env.NEXT_PUBLIC_STRIPE_PK,
})

// Usage in server code
import { serverEnv } from '@/lib/env'
const db = createClient(serverEnv.DATABASE_URL)

// Usage in client code
import { clientEnv } from '@/lib/env'
const stripe = new Stripe(clientEnv.NEXT_PUBLIC_STRIPE_PK)

// ■ NEVER do this — exposes secret to client!
// NEXT_PUBLIC_DATABASE_URL=postgres://...  ← DO NOT!

// ■ This returns undefined in client components:
'use client'
process.env.DATABASE_URL  // ← undefined! (server-only)

// ■ NEXT_PUBLIC_ works in client:
process.env.NEXT_PUBLIC_API_URL  // ← works!
```

■ Interview Tip:
Create a validated env.ts module and import from it everywhere — never use process.env.MY_VAR directly. If a required env var is missing, you'll get a clear error at import time instead of a cryptic undefined error at runtime.

Q36 Deployment — Vercel, Docker, and self-hosting?

■ Answer

Next.js can be deployed to Vercel (easiest), Docker containers, or any Node.js server.

Vercel (recommended for Next.js):

- Zero-config deployment — push to git → auto-deploy
- Automatic preview deployments for PRs
- Edge Network — global CDN for static assets
- Serverless Functions for API routes/SSR
- ISR support out of the box

Docker deployment:

- output: 'standalone' in next.config.js
- Reduces image size — includes only required files
- Use multi-stage Dockerfile

Self-hosting (Node.js server):

- npm run build + npm start
- Requires Node.js on server
- No automatic ISR — need redis/DB for revalidation

Static export:

- output: 'export' in next.config.js
- Generates static HTML/CSS/JS
- No SSR, no API routes, no ISR
- Host on any static host (GitHub Pages, S3, Netlify)

```
// next.config.js for Docker
module.exports = {
  output: 'standalone',  // self-contained output
}

// Dockerfile — optimized multi-stage
// FROM node:20-alpine AS base

// # Dependencies
// FROM base AS deps
// WORKDIR /app
// COPY package*.json ./
// RUN npm ci

// # Builder
// FROM base AS builder
// WORKDIR /app
// COPY --from=deps /app/node_modules ./node_modules
// COPY . .
// ENV NEXT_TELEMETRY_DISABLED 1
```

```
// RUN npm run build
```

```

// # Runner
// FROM base AS runner
// WORKDIR /app
// ENV NODE_ENV production
// ENV NEXT_TELEMETRY_DISABLED 1
// RUN addgroup --system --gid 1001 nodejs
// RUN adduser --system --uid 1001 nextjs
// COPY --from=builder /app/public ./public
// COPY --from=builder --chown=nextjs:nodejs /app/.next/standalone ./
// COPY --from=builder --chown=nextjs:nodejs /app/.next/static ./.next/static
// USER nextjs
// EXPOSE 3000
// ENV PORT 3000
// CMD ['node', 'server.js']

// Static export
// next.config.js
module.exports = {
  output: 'export',
  trailingSlash: true,  // for S3/GitHub Pages
  images: { unoptimized: true }  // required for static export
}
// npm run build → out/ directory with static files

// GitHub Actions CI/CD
// on: push branches: [main]
// jobs:
//   deploy:
//     runs-on: ubuntu-latest
//     steps:
//       - uses: actions/checkout@v3
//       - run: npm ci && npm run build
//       - uses: vercel/action  # or push Docker image
```

■ Interview Tip:
For hobby projects and startups, Vercel is by far the easiest and fastest deployment option — one git push deploys your entire app with optimal CDN distribution. For enterprise or specific compliance requirements, the standalone Docker output with a multi-stage Dockerfile is production-ready.

Q37 Testing Next.js Applications?

■ Answer

Testing a Next.js app requires different strategies for different layers of the application.

Testing pyramid for Next.js:

- Unit tests — components, utilities, server functions
- Integration tests — API routes, database interactions
- E2E tests — full user flows in browser

Tools:

- Jest + React Testing Library — unit/integration
- Vitest — faster Jest alternative
- Playwright or Cypress — E2E tests
- MSW (Mock Service Worker) — mock API calls

Testing Server Components:

- Render async server components with React's server render
- Or test underlying data functions separately
- Server components are harder to test than client components

Testing Client Components:

- React Testing Library — render, query, fire events
- Mock hooks, context, and router

E2E with Playwright:

- Test real user flows
- Test across browsers
- API mocking, visual regression

```
// jest.config.ts
import type { Config } from 'jest'
import nextJest from 'next/jest.js'

const createJestConfig = nextJest({ dir: './' })

const config: Config = {
  setupFilesAfterFramework: ['<rootDir>/jest.setup.ts'],
  testEnvironment: 'jsdom',
  moduleNameMapper: { '^@/(.*)$': '<rootDir>/$1' }
}
export default createJestConfig(config)

// __tests__/components/Button.test.tsx
import { render, screen, fireEvent } from '@testing-library/react'
import { Button } from '@/components/Button'

test('calls onClick when clicked', () => {
```

```
  const handleClick = jest.fn()
```

```
  render(<Button onClick={handleClick}>Click Me</Button>)
  fireEvent.click(screen.getByRole('button'))
  expect(handleClick).toHaveBeenCalledTimes(1)
})

// __tests__/api/users.test.ts — test Route Handler
import { GET, POST } from '@/app/api/users/route'
import { NextRequest } from 'next/server'

test('GET /api/users returns user list', async () => {
  const req = new NextRequest('http://localhost/api/users')
  const res = await GET(req)
  const data = await res.json()
  expect(res.status).toBe(200)
  expect(Array.isArray(data)).toBe(true)
})

// e2e/auth.spec.ts — Playwright E2E
import { test, expect } from '@playwright/test'

test('user can log in', async ({ page }) => {
  await page.goto('/login')
  await page.fill('[name=email]', 'test@example.com')
  await page.fill('[name=password]', 'password')
  await page.click('button[type=submit]')
  await expect(page).toHaveURL('/dashboard')
  await expect(page.getByText('Welcome')).toBeVisible()
})
```

■ Interview Tip:
Test the critical paths with E2E tests (login, checkout, onboarding) and use unit tests for utility functions and complex client components. Skip testing simple Server Components — focus on testing the data functions they call (getProduct, getUser) in isolation.

Q38 SEO Best Practices in Next.js?

■ Answer

Next.js is excellent for SEO because SSR/SSG delivers pre-rendered HTML to crawlers.

SEO fundamentals in Next.js:

- Use SSG or SSR — crawlers see full HTML content
- Metadata API — title, description, og tags
- Structured data (JSON-LD) — rich search results
- sitemap.ts — auto-generated sitemap
- robots.ts — crawl instructions
- Canonical URLs — avoid duplicate content

Core Web Vitals:

- LCP — optimize images, SSG/ISR for fast load
- CLS — set explicit image dimensions, font display:swap
- INP — minimize event handler overhead

Semantic HTML:

- Correct heading hierarchy (one h1 per page)
- Alt text on all images
- aria labels for interactive elements

Technical SEO:

- Canonical tags — prevent duplicate URL indexing
- hreflang — for i18n
- Open Graph — social sharing previews

```
// app/sitemap.ts — auto-generated sitemap
import { MetadataRoute } from 'next'

export default async function sitemap(): Promise<MetadataRoute.Sitemap> {
  const posts = await getPosts()
  const products = await getProducts()

  const staticRoutes = ['/', '/about', '/pricing', '/blog'].map(route => ({
    url: 'https://myapp.com${route}',
    lastModified: new Date(),
    changeFrequency: 'monthly' as const,
    priority: route === '/' ? 1 : 0.8
  }))

  const dynamicRoutes = posts.map(post => ({
    url: 'https://myapp.com/blog/${post.slug}',
    lastModified: post.updatedAt,
    changeFrequency: 'weekly' as const,
    priority: 0.6
  }))
```

```
```

```
  return [...staticRoutes, ...dynamicRoutes]
}

// app/robots.ts
export default function robots() {
  return {
    rules: [
      { userAgent: '*', allow: '/', disallow: ['/api/', '/admin/'] }
    ],
    sitemap: 'https://myapp.com/sitemap.xml'
  }
}

// JSON-LD structured data
export default async function BlogPost({ params }) {
  const post = await getPost(params.slug)
  const jsonLd = {
    '@context': 'https://schema.org',
    '@type': 'Article',
    headline: post.title,
    author: { '@type': 'Person', name: post.author },
    datePublished: post.createdAt,
    image: post.coverImage
  }
  return (
    <>
      <script
        type='application/ld+json'
        dangerouslySetInnerHTML={{ __html: JSON.stringify(jsonLd) }}
      />
      <article>{post.content}</article>
    </>
  )
}
```

■ Interview Tip:
JSON-LD structured data is one of the highest-ROI SEO investments. Adding Article schema to blog posts enables Google rich results (including author info and dates in search). Product schema enables price and availability in search. It's just a JSON object — takes 10 minutes to add.

Q39 Internationalization (i18n) in Next.js?

■ Answer

Next.js has built-in i18n routing and supports library-based translations.

App Router i18n approach:

- Middleware-based locale detection and routing
- [lang] dynamic segment for locale prefix
- Libraries: next-intl, next-i18next, react-i18next

URL patterns:

- /en/about, /fr/about — prefix all routes
- myapp.com (en), fr.myapp.com (fr) — domain-based

next-intl (recommended for App Router):

- Server component support
- Type-safe translations
- Client component support
- Middleware for locale redirect

Pages Router i18n (built-in):

- Configure in next.config.js
- Automatic locale detection
- router.locale, router.locales

```
// ■■ next-intl setup (App Router) ■■

// app/[locale]/layout.tsx
import { NextIntlClientProvider, useMessages } from 'next-intl'

export default function LocaleLayout({ children, params: { locale } }) {
  const messages = useMessages()
  return (
    <html lang={locale}>
      <body>
        <NextIntlClientProvider locale={locale} messages={messages}>
          {children}
        </NextIntlClientProvider>
      </body>
    </html>
  )
}

// messages/en.json
// { 'nav': { 'home': 'Home', 'about': 'About' }, 'hero': { 'title': 'Welcome!' } }

// messages/fr.json
// { 'nav': { 'home': 'Accueil', 'about': 'À propos' }, 'hero': { 'title': 'Bienvenue!' } }

```

```
// app/[locale]/page.tsx — Server Component with translations
```

```
import { useTranslations } from 'next-intl'

export default function HomePage() {
  const t = useTranslations('hero')
  return <h1>{t('title')}</h1>
}

// middleware.ts — locale detection and redirect
import createMiddleware from 'next-intl/middleware'

export default createMiddleware({
  locales: ['en', 'fr', 'es', 'de'],
  defaultLocale: 'en',
  localePrefix: 'always'  // always prefix: /en/about
})

export const config = {
  matcher: ['/((?!api|_next|_vercel|.*\..*).*)']
}

// Generate static pages for all locales
export function generateStaticParams() {
  return [{ locale: 'en' }, { locale: 'fr' }, { locale: 'es' }]
}
```

■ Interview Tip:
next-intl is the gold standard for App Router i18n — it's the only library that properly supports React Server Components, allowing translations in both server and client components with full TypeScript support. Use generateStaticParams to pre-generate pages for all locales at build time.

Q40 Monorepo with Next.js?

■ Answer

A monorepo hosts multiple packages/apps in one repository — sharing code between them.

Monorepo tools:

- Turborepo — build caching, task orchestration (by Vercel)
- Nx — powerful, more opinionated
- pnpm workspaces — package management (use with Turborepo)

Typical monorepo structure:

- apps/web — Next.js frontend
- apps/docs — docs site (also Next.js)
- packages/ui — shared component library
- packages/db — Prisma database layer
- packages/config — shared tsconfig, eslint configs

Benefits:

- Share UI components between multiple Next.js apps
- Share database schemas and queries
- Share types between frontend and backend
- Atomic commits across packages
- Single CI/CD pipeline

Turborepo pipeline:

- Define task dependencies in turbo.json
- Cache build outputs — only rebuild changed packages
- Parallel task execution

```
// Create Turborepo monorepo:
// npx create-turbo@latest

// Project structure:
// ■■■ apps/
// ■   ■■■ web/            ← Next.js app
// ■   ■   ■■■ package.json
// ■   ■   ■■■ next.config.js
// ■   ■■■ docs/           ← Next.js docs
// ■■■ packages/
// ■   ■■■ ui/             ← shared components
// ■   ■   ■■■ package.json  (name: '@repo/ui')
// ■   ■   ■■■ src/Button.tsx
// ■   ■■■ db/             ← Prisma + queries
// ■   ■   ■■■ package.json  (name: '@repo/db')
// ■   ■   ■■■ prisma/schema.prisma
// ■   ■■■ config/         ← tsconfig, eslint
// ■■■ turbo.json
// ■■■ pnpm-workspace.yaml
```

```

```

```
// turbo.json
// {
//   "$schema": "https://turbo.build/schema.json",
//   "pipeline": {
//     "build": { "dependsOn": ["^build"], "outputs": [".next/**"] },
//     "dev": { "cache": false, "persistent": true },
//     "lint": {}
//   }
// }

// apps/web/package.json
// { 'dependencies': { '@repo/ui': '*', '@repo/db': '*' } }

// apps/web/next.config.js — transpile shared packages
module.exports = {
  transpilePackages: ['@repo/ui', '@repo/db']
}

// Usage in Next.js app
import { Button } from '@repo/ui'
import { db } from '@repo/db'

export default async function Page() {
  const users = await db.user.findMany()
  return users.map(u => <Button key={u.id}>{u.name}</Button>)
}

// Build only affected packages:
// turbo build --filter=web  ← only web and its deps
// turbo build --filter=...[HEAD^1]  ← only changed since last commit
```

■ Interview Tip:
Turborepo's remote caching is the killer feature for large teams — build artifacts are cached in the cloud and shared across all developers' machines and CI. If your colleague already built a package with those exact inputs, your machine downloads the output instead of rebuilding.

Next.Js 40/80 Q — Complete! ■

■ All 40/80 questions answered with real code examples!

Next: Complete DSA Sheet to Crack Coding Interviews

■ SAVE — your Testing interview bible

■■ REPOST — help your dev community

■ COMMENT 'JS950' — get all 950 Q PDF

■ FOLLOW Kaushal Singh — daily prep

Kaushal Singh

Full-Stack Dev | MERN + React Native | Next.Js

1M+ Impressions | Testing · Jest · Cypress · Interview Prep
| Next: Complete DSA Sheet to Crack Coding Interviews